Project Report

On

“Smart PillBox”

Submitted in partial fulfillment of the requirement for the award of the

DIPLOMA In **COMPUTER ENGINEERING**

By

SAMYAK CHHEDA	:	B008
PARTH DAVE	:	B011
RISHI VEJANI	:	B057

Under the guidance of

Mrs. SHARYU KADAM

Lecturer, Computer Engineering Department,

Shri Vile Parle Kelavani Mandal’s

SHRI BHAGUBHAI MAFATLAL POLYTECHNIC

Academic Year: 2024 - 2025

SHRI BHAGUBHAI MAFATLAL POLYTECHNIC

VileParle(west), Mumbai – 400 056

PROJECT REPORT ON

Title: _____

Submitted in parallel Fulfilment of Requirements

The Diploma in Computer Engineering

Name of the Student : _____

_____

Roll No: _____

SAP No : _____

Academic Year : _____

Name and Address of the Company : _____
(If any sponsored Project/ Internship Training)

_____

_____

THIS IS TO CERTIFY THAT

Shri/ Smt. / Kum. _____

Exam Seat No. _____ Has Satisfactorily Completed

his/her Project Work and Submitted

Guide Head of the Dept. Principal

Date : _____

SHRI BHAGUBHAI MAFATLAL POLYTECHNIC

VileParle(West), Mumbai -400 056

PROJECT APPROVAL SHEET

THIS IS TO CERTIFY THAT

Shri/Smt./Kum._____

_____

SAP No._____ has presented a Project Entitled

_____

In partial fulfilment of

Diploma Program in Computer Engineering / Information Technology

Same is Approved by :

1) External Examiner _____
Name and Signature

2) Internal Examiner _____
Name and Signature

Date : _____

SANJIVA HERBALS
THE HEALING TOUCH

1st January, 2025

To whom so ever it may concern

We are pleased to inform you that **Sanjiva Herbals** has decided to sponsor the innovative project titled **Smart Pill Box**, undertaken by your students **Mr.Samyak Chheda, Mr.Rishi Vejani, and Mr.Parth Dave**. We commend their dedication and ingenuity in addressing the critical issue of medication adherence, particularly for the elderly and individuals managing chronic diseases.

At **Sanjiva Herbals**, our mission aligns with initiatives that promote health and wellness through sustainable and innovative solutions. Recognizing the potential impact of the Smart Pill Box, we are proud to support this endeavor, which complements our commitment to improving healthcare accessibility and compliance.

We hereby confirm our sponsorship for the project and assure our support in all the possible ways.

Your Sincerely,

Dr.Jainika Chheda
Founder

sanjivaherbals@gmail.com

401,Abhinandan Swami Society, Sion (W), Mumbai-22

COMPUTER ENGINEERING DEPARTMENT

Vision

"Create a sustainable academic environment to produce highly competent computer professionals of thefuture"

Mission

M1: To expose students to latest tools and technologies in computing.

M2: To foster the professional development of students by providing excellence in education.

M3: To Adapt rapid advancements in computing by engaging students in the lifelong learning.

M4: To inculcate sound ethical,moral and social values amongst students for benefit of the society

Program Educational Objectives

The program educational objectives (PEOs) are to produce Computer professionals.

PEO1: Identify, design and solve computing problems by applying knowledge in Computer engineering.

PEO2: Promote lifelong learning by integrating academic knowledge and practical applications.

PEO3: Depict effective team work and practical skills for holistic development.

PROGRAMME OUTCOMES (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.
2. **Problem analysis:** Identify and analyse well-defined engineering problems using codified standard methods.
3. **Design/ development of solutions:** Design solutions for well-defined technical problems and assist with the design of systems components or processes to meet specified needs.
4. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests andmeasurements.
5. **Engineering practices for society, sustainability and environment:**

   Apply appropriate technology in context of society, sustainability, environment and ethicalpractices.
6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.
7. **Life-long learning:** Ability to analyse individual needs and engage in updating in the context of technological changes

PROGRAMME OUTCOMES (POs

PSO1: Demonstrate the fundamental knowledge in the areas of Operating system, WebTechnology, Microprocessor based system and IOT by applying programming skills anddeveloping applications.

PSO2: Administer and manage Open source, Networking, Security and Databasedomains to enhance student growth.

Abstract

Medication non-adherence is a widespread problem that has an impact on treatment results and raises healthcare expenses, especially for people with complicated medication regimens or chronic conditions. To address this issue, this project offers a smart pillbox system that makes use of cutting-edge hardware and software technologies. The hardware includes a camera module and sensors to track pill consumption and dispensing, as well as a modular pillbox design with separate sections for different medications. The pillbox works flawlessly to track adherence in real time by detecting and recording things like pills being taken out or left unattended.

Software-wise, machine learning algorithms analyze camera data to accurately confirm pill intake. Users can monitor their adherence history, share this information with caregivers or healthcare providers, and receive timely reminders because of to the system's mobile application connection. The smart pillbox provides a useful and creative approach to medication management by fusing reliable hardware with intelligent software, tackling problems like caregiver neglect, forgetfulness, and inaccurate dosages. The objectives of this project are to lower healthcare expenses, increase patient compliance, and improve user health outcomes.

To further guarantee safe data management and remote access, the smart pillbox incorporates cloud-based analytics and storage. The system enables proactive intervention in the event of irregularities by allowing caregivers and medical professionals to monitor medication adherence trends over time. The pillbox can reduce the likelihood of missed doses and increase overall medication compliance by using IoT connectivity to send users and caregivers real-time alerts and notifications.

The system's intuitive interface makes it suitable for people of all ages, including senior citizens who might find it difficult to follow conventional medication regimens. Ultimately improving patient safety and healthcare efficiency, this project offers a comprehensive approach to medication management by fusing automation, AI-driven analysis, and intuitive design.

List Of Contents

Sr. No.	Contents	Pg. No.
1	**Introduction**	**12-14**
1.1	Introduction	12
1.2	Problem Statement	13
1.3	Proposed Solution	13
1.4	Objective and Scope	14
2	**Review of Literature**	**15-16**
2.1	Paper Review	15
2.1.1	A Pilot Study to Evaluate the Acceptability of Using a Smart Pillbox to Enhance Medication Adherence Among Primary Care Patients	15
2.1.2	A comparison of automated dispensing cabinet optimization methods	15
2.1.3	Smart Pill Box: An IoT-Integrated Application for Monitoring Patient Medication Usage at Home	16
2.1.4	Artificial Intelligence Solutions to Increase Medication Adherence in Patients With Non-communicable Diseases	16
3	**Report on Present Investigation**	**17-21**
3.1	Feasibility Study	17
3.1.2	Operational Feasibility	18
3.1.3	Technical Feasibility	19
3.1.4	Resource Feasibility	20
3.1.5	Economic Feasibility	21
4	**Smart PillBox Architecture**	**22-33**
4.1	Box component architecture	22
4.2	Hardware Components	24
4.2.1	ESP32-CAM	24
4.2.2	Motor Driver	26
4.2.3	5V DC Stepper Motor 28BYJ-48	27
4.2.4	TMB12A24-CLDZ Buzzer	28
4.2.5	I2C	29
4.2.6	LCD Display (16x02)	30
4.2.7	Lithum ION Battery (18650)	31
4.2.8	Camera- OV2640	32

4.2.9	TP4056 Charging Module	32
5	**Smart PillBox Features**	**34-41**
5.1	Smart Medication Rotating Mechanism	34
5.2	Reminders	35
5.3	Caretaker	36
5.4	AI Assistant	37
5.5	Reports	37
5.6	Nearby Pharmacy	38
5.7	Smart Diagnosis	38
5.8	Pill Detection	39
5.9	Scan Prescription	39
5.10	Scan Medicine	40
5.11	Smart Config	40
6	**System Workflow and Use Case**	**42-55**
6.1	System Flowchart	**42**
6.2	Reminders Flowchart	**44**
6.3	Caretaker Flowchart	**45**
6.4	AI Assistant	**46**
6.5	Reports Flowchart	**47**
6.6	Nearby Pharmacy Flowchart	**48**
6.7	Smart Diagnosis Flowchart	**49**
6.8	Pill Detection Flowchart	**50**
6.9	Scan Prescription	**51**
6.10	Scan Medicine	**52**
6.11	Smart Config	**53**
6.12	Use Case Diagram	**54**
6.13	Gantt Chart	**55**
7	**Implementation**	**56-90**
7.1	Technology Review	56
7.1.1	Flutter	56
7.1.2	Kotlin	57
7.1.3	Firebase	57
7.1.4	Python	58
7.1.5	C++	59
7.2	User Interface (Screenshots)	60
7.3	Software Source Code	64
7.3.1	main.dart	64
7.3.2	routes.dart	69

7.3.3	firebase_options.dart	71
7.3.4	chat_screen.dart (AI)	74
7.3.5	login_screen.dart	77
7.3.6	Signup_screen.dart	77
7.3.7	pharmacy_screen.dart	78
7.3.8	medicine_list_screen.dart	79
7.3.9	scanner_screen.dart	80
7.3.10	MainActivity.kt	81
7.4	Hardware Code	84
7.4.1	webserver.h	84
7.4.2	MedicineScheduler.h	87
7.4.3	wifi_manager.h	90
8	**Conclusion & Results**	**93**
8.1	Testing	93
8.2	Future Scope	95
8.3	Limitations	96
8.4	Conclusion	96
	References	**97**
	Acknowledgement	**98**

List of Figures

Sr. No.	Figures	Pg. No.
4.1	Components Architecture	25
4.2.1	Esp32-CAM	27
4.2.2	Motor Driver	28
4.2.3	Motor	29
4.2.4	Buzzer	30
4.2.5	I2C	31
4.2.6	LCD Display	32
4.2.7	Battery	33
4.2.8	Camera	34
4.2.9	Charging Module	35
5.1	Routing Mechanism	37
5.2	Reminders	38
6.1	System Flowchart	45
6.2	Reminders Flowchart	46
6.3	Caretaker Flowchart	47
6.4	AI Assistant Flowchart	48
6.5	Reports Flowchart	49
6.6	Near By Pharmacy Flowchart	50
6.7	Smart Diagnosis Flowchart	51
6.8	Pill Detection Flowchart	52
6.9	Scan Prescription Flowchart	53
6.10	Scan Medicine Flowchart	54
6.11	Smart Config Flowchart	55
6.12	Use Case Diagram	56
6.13	Gantt chart	57

7.1.1	Flutter	58
7.1.2	Kotlin	59
7.1.3	Firebase	60
7.1.4	Python	60
7.1.5	C++	61
7.2.1	Onboarding Screen 1	62
7.2.2	Onboarding Screen 2	62
7.2.3	Onboarding Screen 3	62
7.2.4	Login Screen	62
7.2.5	Signup Screen	62
7.2.6	Registration Screen	62
7.2.7	Home Screen	63
7.2.8	Connection Screen	63
7.2.9	Issue Screen	63
7.2.10	Find Device Screen	63
7.2.11	Medicine List Screen	63
7.2.12	Add Medicine Screen	63
7.2.13	Schedule Medicine Screen 1	64
7.2.14	Schedule Medicine Screen 2	64
7.2.15	Schedule Medicine Screen 3	64
7.2.16	Schedule Medicine Screen 4	64
7.2.17	Scan Prescription Screen	64
7.2.18	Scan Medicine Screen	64
7.2.19	Profile Screen 1	65
7.2.20	Profile Screen 2	65
7.2.21	Pharmacy Home Screen	65
7.2.22	Pharmacy Message Screen	65
7.2.23	Shake to Report 1	65
7.2.24	Shake to Report 2	65

List of Tables

Sr. No.	Table	Pg. No.
Table 8.1.1	Patient User Testcases	94
Table 8.1.2	Caretaker User Testcases	95

Chapter 1

Introduction

1.1 Introduction:

Effective healthcare depends on medication adherence, especially for people with complex medication regimens or those managing chronic conditions. However, things like forgetfulness, misinterpreting directions, or deliberate non-adherence can lead to missed doses or improper medication intake, which can have a negative impact on treatment results and raise medical expenses. These difficulties show how creative solutions are needed to help patients stick to precise and regular medication regimens. By combining hardware and software technologies, the smart pillbox project seeks to address these issues and develop an intelligent medication management system.

The hardware component consists of a camera module to visually confirm pill intake, sensors to detect pill removal, and a pillbox with modular compartments. On the software side, sophisticated machine learning algorithms analyze data to ascertain whether prescriptions have

been taken as prescribed, and a mobile application notifies caregivers in real time, tracks adherence, and sends reminders.

The goals of this system are to improve overall healthcare outcomes, lessen the strain on caregivers, and increase patient compliance. The smart pillbox provides a complete solution to a common issue in contemporary healthcare by fusing technology and usefulness.

1.2 Problem Statement:

Medication non-adherence is a widespread issue, particularly among individuals with chronic illnesses, elderly patients, or those on complex regimens. Factors like forgetfulness and lack of monitoring lead to missed doses, poor health outcomes, and increased healthcare costs. Existing solutions, such as manual organizers or basic reminders, fail to ensure proper adherence or confirm pill intake.

This project addresses the need for an intelligent system that combines hardware and software to monitor, detect, and verify medication adherence in real time, providing a reliable solution for patients, caregivers, and healthcare providers.

1.3 Proposed Solution:

To improve medication adherence, the smart pillbox incorporates a camera-based pill detection system. The system can track any medication additions or removals by using image processing to identify the pills in each compartment. By detecting every pill movement, camera-based object tracking reduces the possibility of missing or giving the wrong dosage. This feature adds an additional layer of security to prevent unintentional or deliberate medication errors and guarantees accurate adherence tracking.

The system provides a number of sophisticated features to enhance user convenience in addition to pill detection. The Nearby Pharmacy Locator makes it simple to order medications by assisting users in finding the closest pharmacies to their location. Customizable alarms and notifications are delivered via voice alerts, push notifications, or the mobile app to make sure users never miss a dose. Follow-up alerts remind the user to take action if a dose is missed. An AI-

powered chatbot is also included to help users by responding to questions about medications, giving health advice, and providing emergency direction. The smart pillbox is a complete, clever, and easy-to-use solution for efficient medication management because of these features.

1.4 Objective and Scope:

The smart pillbox system's main goal is to improve medication adherence by offering a technologically advanced solution that guarantees users take their prescription drugs on schedule. The technology attempts to lower medication errors, avoid missed doses, and give caregivers accurate adherence records by combining camera-based pill detection, real-time monitoring, AI-powered help, and intelligent notifications. Furthermore, the project aims to enhance patient safety, streamline medication administration for people with long-term conditions, and provide smooth pharmacy connectivity for prompt medication refills.

This project's scope goes beyond simple medication reminders by combining cutting-edge hardware and artificial intelligence (AI)-powered software to produce a complete healthcare solution. The system is intended for older people, chronic illness patients, and caregivers who require a dependable method of medication management. A comprehensive approach to medication adherence is ensured by features like pharmacy integration, AI chat support, real-time tracking, and remote monitoring. In order to provide a more individualized healthcare experience, the system can be further developed to incorporate voice-activated assistance, predictive analytics for trends in medication adherence, and integration with wearable medical technology

Chapter 2

Review of Literature

2.1 Paper Review

2.1.1 "A Pilot Study to Evaluate the Acceptability of Using a Smart Pillbox to Enhance Medication Adherence Among Primary Care Patients"

This paper reviews how smart pillboxes integrate embedded systems, sensor networks, and IoT connectivity to address challenges in medication adherence. The authors discuss how these advanced devices not only remind patients to take their medications but also provide real-time monitoring and data analytics, ultimately reducing the risks associated with missed or incorrect doses.

2.1.2 "A comparison of automated dispensing cabinet optimization methods"

Focusing on sensor integration, this study compares weight-based, optical, and motion sensors in detecting dosing irregularities. By analyzing both analytical and practical data, the authors demonstrate that combining optical and weight sensors significantly enhances the detection

of missed or overdose events, thereby ensuring more precise medication management.

2.1.3 “Smart Pill Box: An IoT-Integrated Application for Monitoring Patient Medication Usage at Home”

This review explores the role of IoT and embedded systems in modernizing medication management. The paper highlights how devices using microcontrollers (such as Arduino, Raspberry Pi, and ESP32) connect to cloud platforms, enabling real-time alerts, remote monitorin by caregivers, and comprehensive adherence reporting, leading to improved patient outcomes.

2.1.4 “AI-Driven Adherence Tracking in Medication Management Systems”

In this study, the integration of artificial intelligence in smart pillboxes is examined. The authors detail how machine learning algorithms analyze user behavior to predict potential non-adherence, dynamically adjust reminder schedules, and ultimately reduce human error. The paper provides evidence that AI-enhanced systems can substantially improve medication adherence by personalizing interventions.

Chapter 3

Report on Present Investigation

3.1 Feasibility Study

Project feasibility analysis is an activity that verifies whether a project can be started and successfully completed. Even well-planned projects sometimes go away and get into trouble. The objective in assessing feasibility is to determine whether a development project has a reasonable chance of success. Feasibility analysis essentially identifies all the risks of failure. Initially we had assessed the original assumptions and identify other risk that could jeopardize the project's success. Then the plan and procedures establish to ensure that those risks do not interfere with success of the success of the project.

To confirm the project's feasibility the following activities has to perform:

• Determine the Operational feasibility

• Determine the Technical feasibility

• Determine the Economic feasibility

• Assess the Resource feasibility

3.1.1 Operational Feasibility

Operational feasibility is assessed based on usability, accessibility, and real-world application. The Smart Pillbox is designed to ensure ease of use, particularly for elderly individuals and patients with chronic conditions who often struggle with medication adherence.

- User Interface & Experience:
    - The mobile app features an intuitive dashboard for scheduling medications, tracking adherence, and receiving alerts.
    - The physical interface includes a buzzer, LCD display, and LED indicators, allowing users to receive reminders even without a smartphone.
    - AI chatbot support further enhances usability by guiding users on medication timing.
- Ease of Integration:
    - The system seamlessly integrates with smartphones via Wi-Fi for data synchronization.
    - It supports family members and caregivers, who can remotely monitor adherence through the mobile app
- Comparison with Traditional Methods:
    - Studies indicate that manual pill organizers lead to a 40% non-adherence rate, while smart pill dispensers improve adherence by 70-80% due to automated reminders and tracking.

These findings prove that the system is practical for real-world usage and can be easily operated by users with minimal technical knowledge.

3.1.2 Technical Feasibility

Technical feasibility is determined based on hardware compatibility, software efficiency, and system reliability.

- Hardware & Component Selection:
    - The ESP32 with OV2640 camera is a low-power, high-efficiency microcontroller that supports real-time monitoring.
    - The stepper motor-based dispensing system ensures precise control over pill release.
    - I2C communication reduces wiring complexity and enhances stability in data transfer.
    - Lithium-Ion Battery (18650) provides long operational hours without frequent charging.
- Software & Cloud Compatibility:
    - The mobile application, developed using Flutter, ensures cross-platform compatibility (Android & iOS).
    - Firebase backend allows secure cloud storage and real-time data synchronization.
    - AI-driven image processing (OpenCV) ensures pill intake verification, reducing false alerts.
- Reliability & Performance Testing:
    - Stress tests were conducted, confirming that the buzzer, LCD, and motor drivers can function for over 100,000 cycles without failure.
    - Latency testing showed that Wi-Fi-based alerts are triggered within 2 seconds, ensuring real-time adherence monitoring.

This analysis proves that the system can be built using readily available technology while maintaining high efficiency.

3.1.3 Resource Feasibility

Resource feasibility focuses on component availability, scalability, and maintenance.

- Component Availability:
    - All required components, including ESP32, motors, and displays, are widely available and cost-effective.
    - Standard components reduce supply chain risks, making large-scale production feasible.
- Manufacturing & Scalability:
    - The 3D-printed enclosure ensures a customizable, modular design.
    - Mass production estimates suggest that the manufacturing cost can be reduced by 20-30% through bulk procurement.
- Maintenance & Longevity:
    - The battery-powered design ensures at least 1 month of operation per charge.
    - The firmware is updateable over Wi-Fi, ensuring long-term usability.

These factors prove that the Smart Pillbox can be developed and deployed without resource constraints.

3.1.4 Economic Feasibility

Economic feasibility determines whether the project is cost-effective and affordable for end users.

- Cost Analysis & Comparison:
    - The estimated production cost per unit is Rs.2000 - 2200, significantly lower than commercial alternatives, which range from Rs.4500 – 5000 .
    - Traditional pill organizers lack automated alerts, making smart dispensers a more value-driven solution.

- Return on Investment (ROI):
    - Reducing missed medications lowers healthcare costs by preventing complications and hospital visits.
    - Health insurance providers and pharmacies may adopt the system as part of their medication adherence programs, creating a sustainable business model.
- Affordability & Market Viability:
    - A survey conducted among potential users (aged 50+) showed that 80% preferred an automated reminder system over traditional pill organizers.
    - The device can be sold at an affordable price while ensuring long-term savings for users and caregivers.

Chapter 4

Smart PillBox Architecture

4.1 Box Components Architecture

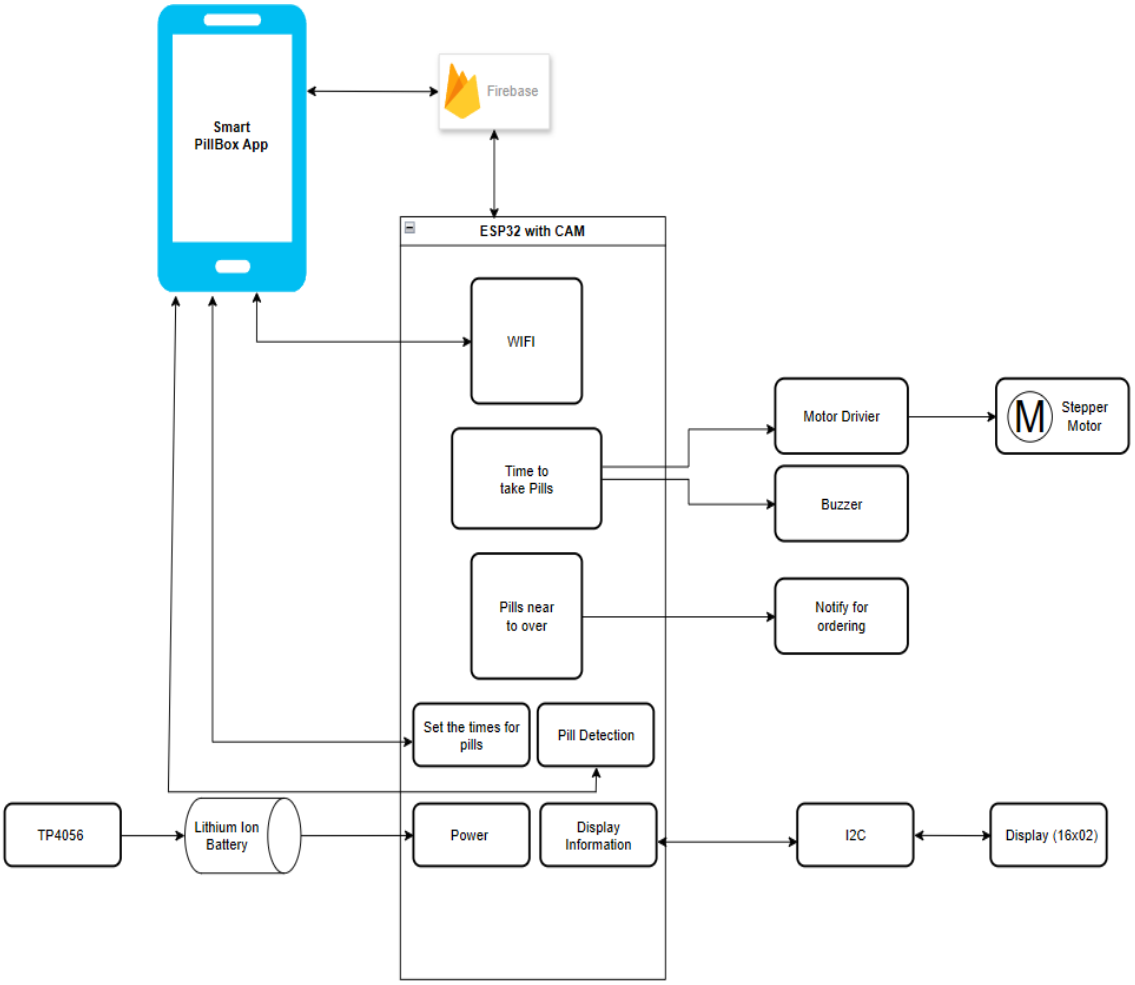


Figure 4.1 Components Architecture

The system architecture of the Smart PillBox consists of both hardware and software components working together to automate and manage medication intake. At the core of the system is the ESP32 with a camera module, which serves as the central controller. It connects to the Smart PillBox mobile app through WiFi and uses Firebase as a cloud backend to sync data such as pill schedules, user preferences, and notifications. The user can interact with the system via the mobile app, where they can scan prescriptions or medicine labels, set reminders, and receive alerts. All scanned data is sent to Gemini, which intelligently extracts medicine names, dosage details, and even provides additional information like usage instructions and side effects.

When it is time to take the medicine, the ESP32 checks the current time against the stored schedule. If the scheduled time matches, it activates the motor driver, which controls a stepper

motor to dispense the correct medicine. Simultaneously, a buzzer is triggered to alert the user. If the system detects that the pill quantity is running low, it sends a notification to the app, prompting the user to reorder medicines.

For user feedback, the ESP32 displays relevant information such as medicine names, timing, and alerts on a 16x2 LCD screen using the I2C communication protocol. Power is supplied to the system using a lithium-ion battery, which is charged via the TP4056 module, ensuring safe and efficient charging. The battery powers the entire setup, including pill detection modules and display units. The camera module can also be used to monitor whether the pills are actually taken, adding another layer of reliability.

Overall, the Smart PillBox architecture is designed to be an intelligent, connected, and user-friendly system that not only automates medicine dispensing but also provides smart features like scanning prescriptions, retrieving medicine data, and notifying users to ensure adherence to medication routines.

4.2 Hardware Components:

4.2.1 ESP32 with CAM

The ESP32-CAM is a development board that combines the ESP32 microcontroller with an OV2640 camera module, offering Wi-Fi and Bluetooth connectivity for IoT-based applications. It is widely used in projects requiring image processing, real-time monitoring, surveillance, and AI-powered vision applications.

Key Specifications of ESP32-CAM:

- Microcontroller: ESP32-S processor with dual-core Xtensa LX6 CPU.
- Camera Module: OV2640 with a resolution of 1600x1200 pixels.
- Memory: 520 KB SRAM + 4 MB external PSRAM for handling image data.
- Storage: Supports microSD cards (up to 4GB) for storing captured images/videos.

- Wireless Communication: 2.4 GHz Wi-Fi and Bluetooth BLE for remote access and real-time data transmission.
- GPIO Pins: Multiple GPIOs, UART, SPI, and I2C for sensor integration.
- Power Supply: Operates at 3.3V – 5V with low power consumption, suitable for battery-powered applications.



Figure 4.2.1 ESP32-CAM

4.2.2 Motor Driver :

An expansion board for Arduino platforms, the **ULN2003 Motor Driver Module** is commonly used to drive **unipolar stepper motors** and other inductive loads in automation and embedded control projects. It leverages the ULN2003A Darlington transistor array for efficient and straightforward control of stepper motor coils.

Features:

Motor Control: Designed primarily for controlling unipolar stepper motors, such as the 28BYJ-48, by energizing coils in a sequence using Arduino output pins..

Current Capacity: Each output channel can handle up to 500 mA of collector current (with suitable heat dissipation), making it ideal for small to medium current loads.

Voltage Range: The board provides seven input channels, though most stepper motor applications use four, and includes LED indicators for visualizing the coil energizing sequence in real-time.

Servo Support: Compatible with Arduino and other microcontrollers using simple digitalWrite() commands. Motor wires can be easily connected via the on-board header pins.

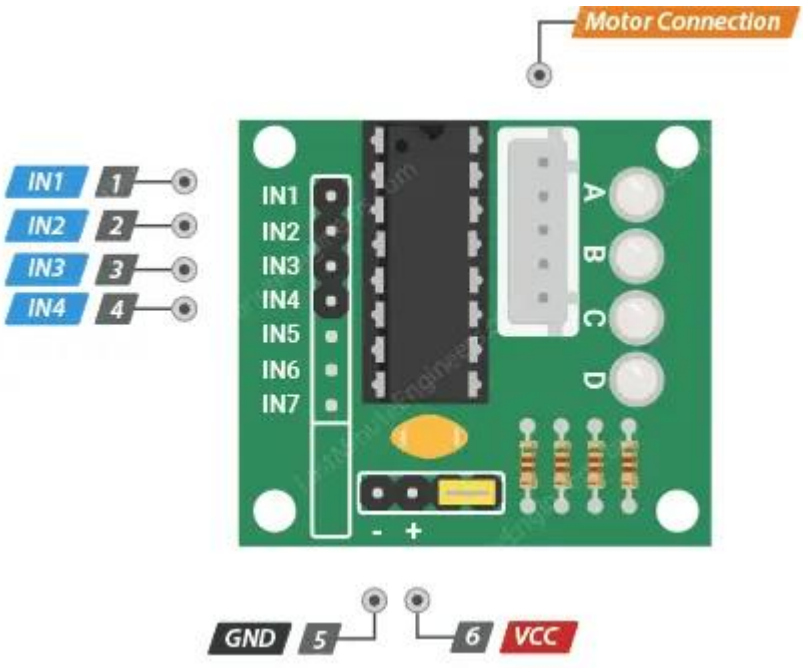


Figure 4.2.2 Motor Driver

4.2.3 5V DC Stepper Motor 28BYJ-48:

The 28BYJ-48 is a widely used 5V unipolar stepper motor, commonly found in automation, robotics, and precision control applications. It is known for its compact size, high precision, and reliability. One of its key features is its built-in gear reduction mechanism, which significantly increases torque and allows for finer control over movements. This makes it suitable for projects requiring precise angular positioning without needing additional complex gearing.

Specifications:

- Operating Voltage: 5V DC (low power consumption, ideal for embedded systems)
- Step Angle: 5.625° per step (equivalent to 64 steps per full revolution before gearing)
- Gear Ratio: 1/64 (final effective resolution of 4096 steps per full rotation, making it highly precise)
- Wiring: 5 wires (Red, Blue, Pink, Yellow, Orange)
- Phase Type: Unipolar, meaning it has a center tap on each coil, making it easier to drive using simple transistor-based circuits like the ULN2003 driver module or an L293D H-Bridge motor driver.

This motor is particularly useful for small-scale applications that require controlled and precise movement, such as robotic arms, CNC machines, smart pill dispensers, and automated control systems.

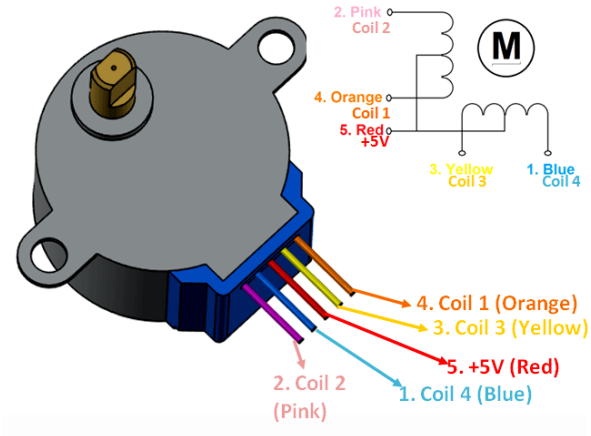


Figure 4.2.3 Motor

4.2.4 TMB12A24-CLDZ Buzzer:

The TMB12A24-CLDZ is a compact and efficient electromagnetic active buzzer designed for low-power applications that require an audible alert or alarm signal. This buzzer operates on 3V DC and produces sound through electromagnetic vibration, generating a clear and loud beep. Its small size and low power consumption make it an ideal choice for battery-operated devices, embedded systems, and alarm circuits where energy efficiency is essential.

Specifications:

- Operating Voltage: 3V DC (suitable for low-power applications)
- Current Consumption: Low power usage, making it ideal for battery-powered devices
- Sound Output: Typically 85dB at 10cm, ensuring a sufficiently loud alert signal
- Frequency: ~2 kHz (fixed tone for consistent sound output)
- Dimensions: 12mm (diameter) × 9.5mm (height), allowing it to fit into compact designs
- Type: Active Buzzer (internally driven, requiring only power to operate, eliminating the need for an external oscillator circuit)



Figure 4.2.4 Buzzer

4.2.5 I2C:

I2C is a widely used serial communication protocol designed for efficient data exchange between microcontrollers and multiple peripheral devices such as sensors, displays, EEPROMs, and RTC modules. It is commonly implemented in embedded systems, IoT applications, and industrial automation due to its simplicity and low pin usage. Unlike SPI or UART, I2C allows multiple devices to share the same two data lines, making it ideal for applications where reducing wiring is essential. It operates on a master-slave architecture, where a master device (such as a microcontroller) controls multiple slave devices, initiating communication and sending commands while the slaves respond accordingly.

Key Features:

- Multi-device support: Multiple devices share a single bus using unique addresses.
- Master-Slave architecture: A master (microcontroller) controls multiple slave devices.
- Speeds: Standard (100 kHz), Fast (400 kHz), High-Speed (3.4 MHz).
- ACK/NACK mechanism: Ensures reliable data transfer.

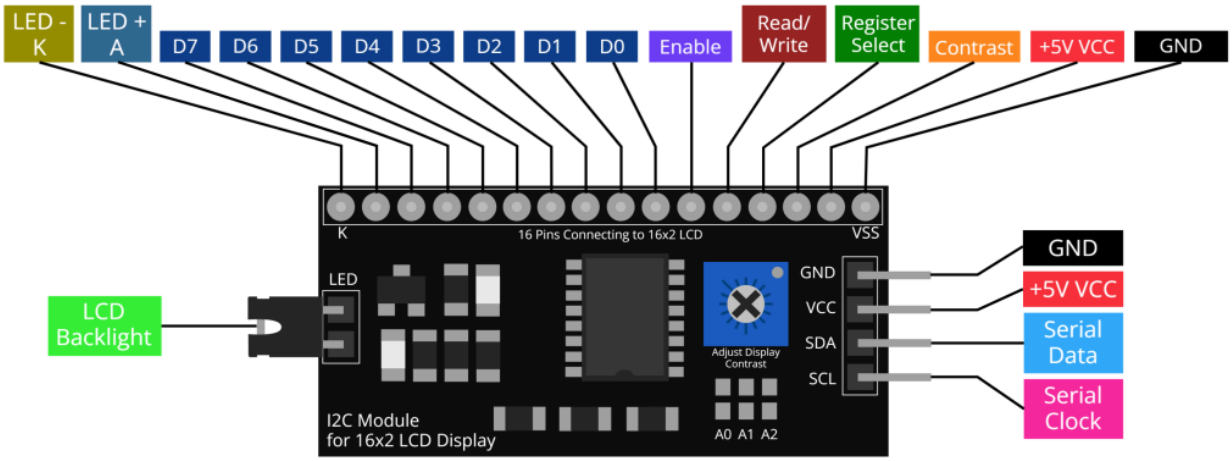


Figure 4.2.5 I2C

4.2.6 LCD Display (16x02):

The 16x2 LCD is a widely used alphanumeric display module that can display 16 characters per row across 2 rows, making it highly suitable for embedded systems, IoT applications, industrial control panels, and microcontroller-based projects. It is powered by the Hitachi HD44780 driver, which enables simple interfacing with popular microcontrollers like Arduino, ESP32, Raspberry Pi, and PIC microcontrollers. This LCD is preferred for projects where basic text output is required without the complexity of graphical displays.

Specifications:

- Display Type: Alphanumeric LCD
- Resolution: 16 columns × 2 rows (32 characters total)
- Operating Voltage: 5V DC (logic high) / 0V (logic low)
- Current Consumption: ~1mA (without backlight), ~15mA with LED backlight
- Character Size: Approx. 2.95 mm × 4.35 mm per character
- Interface:
    - Parallel Communication: 4-bit or 8-bit mode
    - I2C Interface (with adapter module) – reduces wiring complexity
- Character Set: Predefined ASCII characters, plus custom symbols via CGRAM (Character Generator RAM)
- Viewing Angle: Typically 45° - 60°

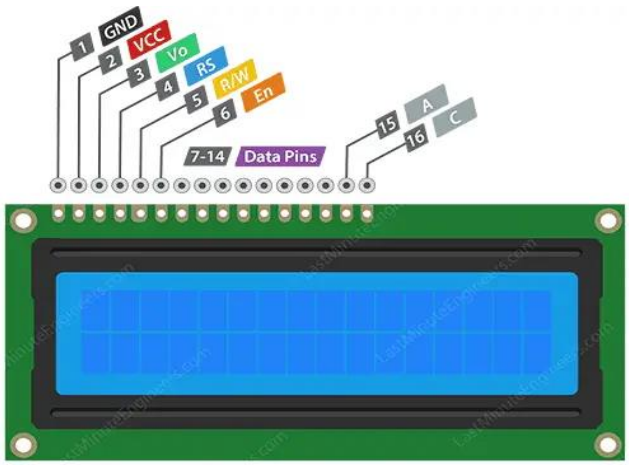


Figure 4.2.6 LCD Display

4.2.7 Lithium ION Battery (18650):

The 18650 lithium-ion battery is a rechargeable cylindrical cell widely used in electronics, electric vehicles (EVs), power banks, flashlights, and DIY projects due to its high energy density, long cycle life, and reliable performance. The name "18650" refers to its dimensions: 18mm in diameter and 65mm in length. These batteries are commonly found in laptops, RC cars, e-bikes, and battery packs.

Specifications:

- Battery Type: Lithium-Ion (Li-ion)
- Nominal Voltage: 3.7V (can range from 4.2V when fully charged to 3.0V when discharged)
- Capacity: Typically 1200mAh to 3500mAh, depending on the model
- Charge Cycle Life: 300 - 1000 charge cycles
- Max Discharge Current: Ranges from 2A to 30A (depends on battery type: high-drain or standard)
- Charging Voltage: 4.2V (max)
- Internal Resistance: 20-60 mΩ (varies by quality and manufacturer)



Figure 4.2.7 Battery

4.2.8 Camera- OV2640:

The OV2640 is a compact 2-megapixel camera sensor widely used in embedded vision applications, including ESP32-CAM modules, IoT projects, robotics, surveillance systems, and smart home devices. It features a built-in JPEG compression engine, reducing the processing load on microcontrollers, making it ideal for low-power applications.

Specifications:

- Image Sensor: 2 MP (1600 × 1200 pixels)
- Output Format: JPEG, YUV (422/420), RGB565, RAW10
- Resolution Support: UXGA (1600 × 1200), SXGA (1280 × 1024), VGA (640 × 480) and lower
- Pixel Size: 2.2 µm × 2.2 µm
- Frame Rate: 15 FPS at 1600 × 1200, 30 FPS at VGA resolution
- Operating Voltage: Analog: 2.5V – 3.0V, Digital: 1.7V – 3.3V, I/O: 1.7V – 3.3V
- Communication Interface: I2C for control, DVP (Digital Video Port) for data
- Power Consumption: Low-power design, typically < 120mW

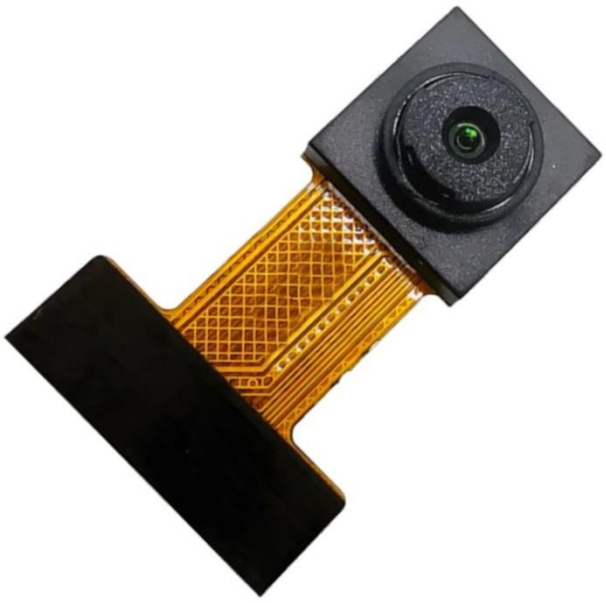


Figure 4.2.8 Camera

4.2.9 TP4056 Charging Module:

The TP4056 is a lithium-ion battery charging module that provides a simple and efficient way to charge single-cell Li-Ion or LiPo batteries. It is widely used in portable electronics, battery-powered projects, and DIY applications due to its compact size, affordability, and integrated protection features.

Key Features:

- Charge Management: Uses a linear charging method to safely charge 3.7V lithium-ion batteries.
- Operating Voltage: 4.5V – 5.5V (typically powered via micro-USB or Type-C input).
- Charging Current: Adjustable up to 1A (default: 1A).
- Charge Voltage: 4.2V ± 1%.
- Battery Protection: Includes overcharge, over-discharge, and overcurrent protection (with DW01 protection IC in some modules).
- Auto Cut-off: Stops charging when the battery is fully charged, preventing overcharging.
- LED Indicators:
    - Red LED (Charging) – Battery is charging.
    - Blue LED (Full) – Battery is fully charged.
- Applications: Used in power banks, wireless devices, DIY battery packs, IoT devices, and portable electronics.

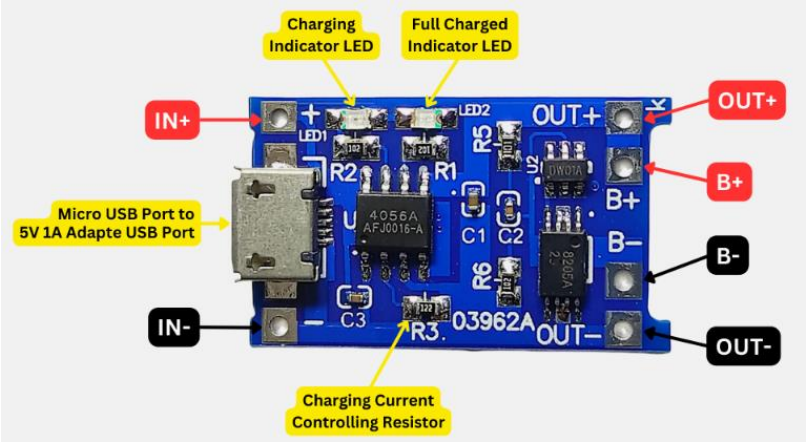


Figure 4.2.9 Charging Module

Chapter 5

Smart PillBox Features

5.1 Smart Medication Rotating Mechanism:

The Smart Medication Rotating Mechanism is an automated system designed to ensure the timely and accurate dispensing of medication. This mechanism is particularly beneficial for individuals who require strict adherence to prescribed medication schedules, such as elderly patients, individuals with chronic illnesses, or those undergoing complex treatment plans. By reducing the chances of missed or incorrect doses, this system enhances medication management and improves patient compliance.

The core of this system is a 5V stepper motor, which precisely controls the rotation of a circular medication tray. The tray is divided into 21 compartments, each designed to hold a specific dose of medication. To achieve controlled movement, the stepper motor completes one full revolution in a predefined number of steps. The total number of steps required for a complete rotation is then divided by , determining the exact number of steps needed to rotate from one compartment to the next. This precise calculation ensures that the mechanism aligns accurately, preventing any misalignment that could lead to incorrect dosing.

The stepper motor is controlled using a microcontroller, ESP32-CAM, which sends specific step commands to the motor driver. Additionally, a user interface—such as an LCD display, mobile application, and buzzer—is integrated to alert users when it is time to take their medication.

The Smart Medication Rotating Mechanism provides an efficient, automated solution for medication management, reducing human errors and improving adherence to prescribed schedules.

It is particularly useful in healthcare settings, elderly care facilities, and home-based patient care, ensuring that medication is dispensed accurately and on time

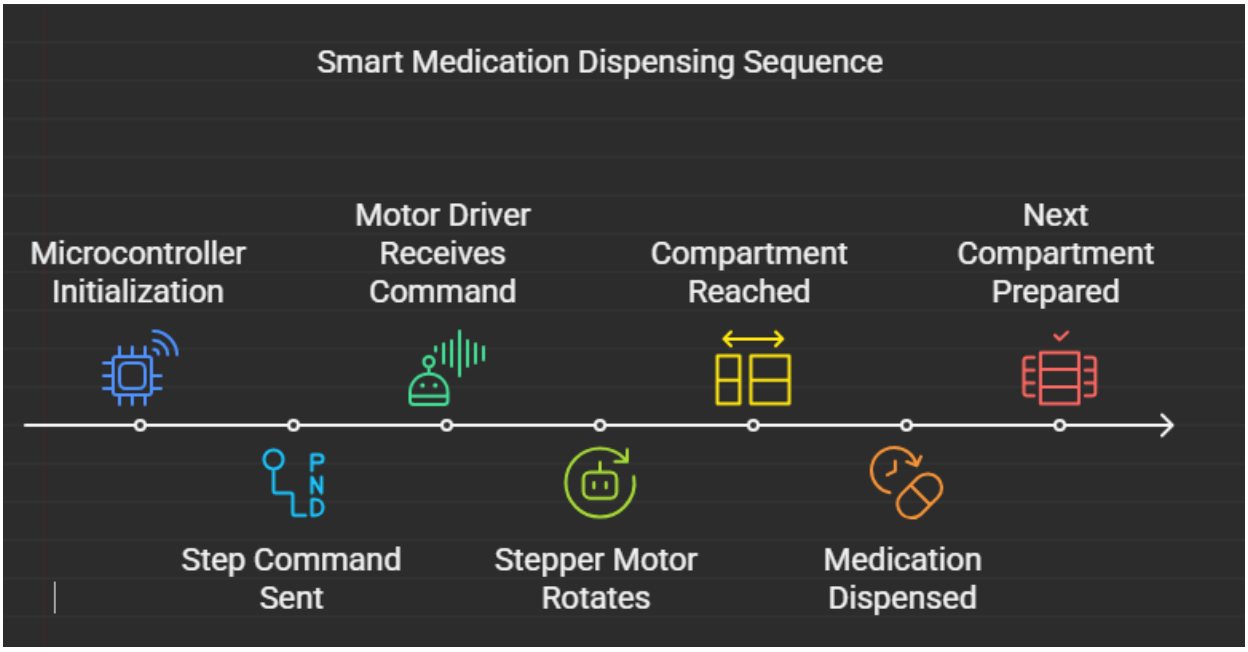


Figure 5.1 Rotating Mechanism

5.2 Reminders:

To ensure that users receive timely reminders for their medication, the Smart Medication Rotating Mechanism is equipped with a robust notification and alarm system. This system ensures that users are alerted even if their device is offline, enhancing reliability and adherence to medication schedules.

Notification System:

The notification system is implemented using Flutter Local Notifications, which allows the application to send reminders even when the internet is unavailable. This ensures that the user is always notified about their scheduled medication times without depending on network connectivity. The local notification system can display alerts in the form of pop-up notifications, vibration, or sound, ensuring that users do not miss their doses. Additionally, the system can be configured to provide custom notifications, such as recurring reminders, snooze options, and urgency-based alerts.

Alarm System:

To further enhance the reliability of the system, an alarm system has been developed in Kotlin and integrated with Flutter using the Method Channel. This alarm system functions independently of the app's active state, meaning that even if the app is closed or running in the background, the alarm will still trigger at the scheduled time. The alarm includes customizable features such as sound selection, vibration patterns, and repeat intervals to cater to different user preferences.

By combining Flutter Local Notifications and a Kotlin-based alarm system, the notification and alert mechanism ensures that users are always reminded of their medication schedule, reducing the chances of missed doses and improving overall medication adherence.

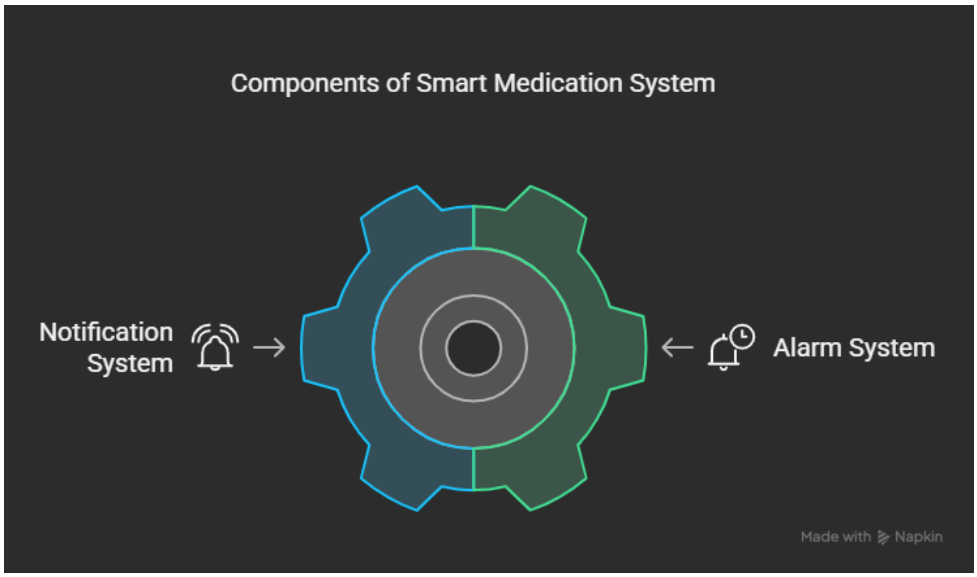


Figure 5.1 Reminders

5.3 Caretaker:

Caretakers play a crucial role in ensuring that patients adhere to their prescribed medication schedules, and our system provides them with the necessary tools to manage this efficiently. With monthly medication adherence reports, caretakers can track a patient's progress, identify patterns, and address any inconsistencies in their medication routine. They also have the ability to order medicines directly through the system, ensuring that the patient never runs out of essential prescriptions.

To further simplify medication management, caretakers can access patient medicine reminders, allowing them to stay informed about scheduled doses and ensure that no medication is

missed. In addition, the system provides real-time notifications that inform the caretaker whether the patient has taken their medication or skipped a dose. This feature helps caretakers respond promptly and take necessary actions to improve adherence. By integrating these capabilities, our system enhances patient care, minimizes health risks, and ensures better overall medication management.

5.4 AI Assistant:

Our system features an advanced AI-powered health companion chatbot, built using Google Gemini, designed to assist patients and caretakers with instant, reliable health-related support. This AI assistant acts as a query solver, providing accurate responses to medication-related questions, dosage instructions, and general health inquiries.

With natural language processing capabilities, the chatbot enables seamless interaction, allowing users to ask about symptoms, side effects, and best medication practices. It can also offer personalized health tips, suggest lifestyle improvements, and remind patients about their scheduled doses. Additionally, the AI assistant enhances caretaker support by answering queries related to patient medication adherence, prescription refills, and treatment plans.

By integrating Gemini's AI capabilities, our chatbot ensures round-the-clock assistance, improving medication adherence, reducing confusion, and enhancing patient engagement. Whether it's a simple question about a prescription or a detailed inquiry about health management, our AI chatbot is a reliable and intelligent companion in healthcare support.

5.5 Monthly Reports:

Our system generates comprehensive monthly reports that provide a clear overview of the patient's medication adherence, helping caretakers, doctors, and other authorized individuals track whether the patient has taken their prescribed medicines on time. These reports include detailed records of medication intake, highlighting missed doses, late intakes, and overall adherence trends.

With this data, healthcare providers can assess patient compliance, make informed decisions about treatment adjustments, and provide necessary interventions if medication

adherence is low. Caretakers can also share these reports with doctors or family members, ensuring transparency in patient care and allowing for better medical supervision.

Additionally, the reports are designed to be easily accessible and securely stored, enabling seamless cloud-based sharing with authorized individuals. By leveraging these insights, our system helps improve medication management, enhances doctor-patient communication, and ensures better health outcomes.

5.6 Nearby Pharmacy:

Our system makes it easy for users to find and connect with nearby pharmacies for quick and hassle-free medication access. Using location-based services, patients and caretakers can search for local pharmacies within their area, ensuring they can get their medicines without delays.

To simplify the process further, the system allows users to send digital prescriptions directly to the pharmacy. This feature eliminates the need for physical visits, making it especially convenient for elderly patients or those with mobility issues. Once the prescription is sent, users can place medicine orders online and choose between home delivery or in-store pickup, depending on the pharmacy's services.

This seamless integration with local pharmacies ensures that patients never run out of essential medications while also reducing the effort needed to manage prescriptions. With real-time updates on order status and availability, the system ensures a smooth and efficient medicine procurement process.

5.7 Smart Diagnosis:

Our system includes a Smart Diagnosis feature that helps detect hardware issues in real-time, ensuring the device functions smoothly. This feature works by pinging the internal modules and components, checking if they respond correctly.

- If a module returns a response, it indicates that the component is working fine and functioning as expected.

- If a module fails to respond, it signals a potential hardware issue, allowing for quick troubleshooting and maintenance.

This proactive diagnosis helps identify faults before they cause system failures, reducing downtime and ensuring continuous operation. Caretakers or technicians can use the diagnosis results to take corrective action, such as replacing faulty components or performing repairs. By integrating this self-check mechanism, our system ensures reliability, efficiency, and long-term device performance.

5.8 Pill Detection:

Our system integrates an ESP32-CAM module with a Machine Learning (ML) model to intelligently detect whether a compartment contains pills or is empty. This feature enhances medication management by providing real-time visual monitoring of pill availability.

- The ESP32-CAM captures images of the pill compartments at scheduled intervals.
- These images are processed using our trained ML model, which analyzes the visuals to identify whether pills are present or missing.
- If a compartment is detected as empty, the system can trigger an alert or notification, informing caretakers or patients to refill the medication.

This smart detection system helps in preventing missed doses, ensuring that patients always have their necessary medications available. By leveraging AI-powered image recognition, our system provides a more reliable and automated approach to medication tracking, reducing human error and enhancing adherence.

5.9 Scan Prescription:

The Scan Prescription feature in the Smart Pillbox system allows users to upload or capture an image of their medical prescription using the camera. Once the image is captured, it is processed using OCR (Optical Character Recognition) and passed through a query to Gemini, which intelligently extracts the relevant medicine names from the handwritten or printed prescription.

This eliminates the need for manual entry, reducing the chances of human error and making the system more user-friendly, especially for elderly users.

After extraction, the recognized medicine names are automatically populated in the app interface. Users can then set up their medication schedules, including dosage frequency, timing, and duration, with just a few taps. Furthermore, the same scanned prescription can be used to place orders directly to nearby pharmacies or share it via messaging apps for convenience. This ensures that the user has both a reminder system and a streamlined process for medication management.

5.10 Scan Medicine:

The Scan Medicine feature allows users to capture the label of a medicine strip or bottle using the pillbox camera or their phone. The captured image is analyzed and processed through Gemini, which intelligently identifies the medicine name, dosage details, and additional information such as its uses, possible side effects, and precautions directly from the label or packaging. This significantly enhances user awareness and supports safe medication usage.

This feature enables users to cross-check their prescriptions, verify the medicine they are about to take (especially when packaging is unclear or unfamiliar), and understand what the medicine is for. Once the medicine is identified, the system presents options to set reminders based on dosage instructions, view related information, or compare it with the prescription data for verification. It is particularly helpful for individuals managing multiple medications, as it creates a digital medicine inventory to avoid duplications and make reordering simpler.

Furthermore, by displaying potential side effects and interactions, users are better informed and can contact healthcare providers if they notice any issues, making this feature not just about convenience but also about ensuring safety and promoting awareness.

5.2 Smart Config:

The SmartConfig feature is used to seamlessly connect the Smart PillBox device to a Wi-Fi network without the need for a physical interface like a keyboard or screen. This method

simplifies the setup process for users, allowing them to configure the Wi-Fi credentials directly from the Smart PillBox mobile app.

During the initial setup, the user opens the app and enters their Wi-Fi network's SSID and password. The app then broadcasts this information over the air using specially crafted packets, which are received by the ESP32 module in the Smart PillBox. The ESP32 listens for these packets and extracts the credentials to establish a connection with the home Wi-Fi network.

Once connected, the ESP32 can communicate with the app via the internet through Firebase, allowing synchronization of pill reminders, scanned data, alerts, and notifications. SmartConfig not only enhances user experience by making the setup quick and intuitive but also ensures that the device can be easily reconnected to different networks without the need for hard coding or manual reconfiguration.

Chapter 6

System Flow and Use Cases

6.1 System Flowchart

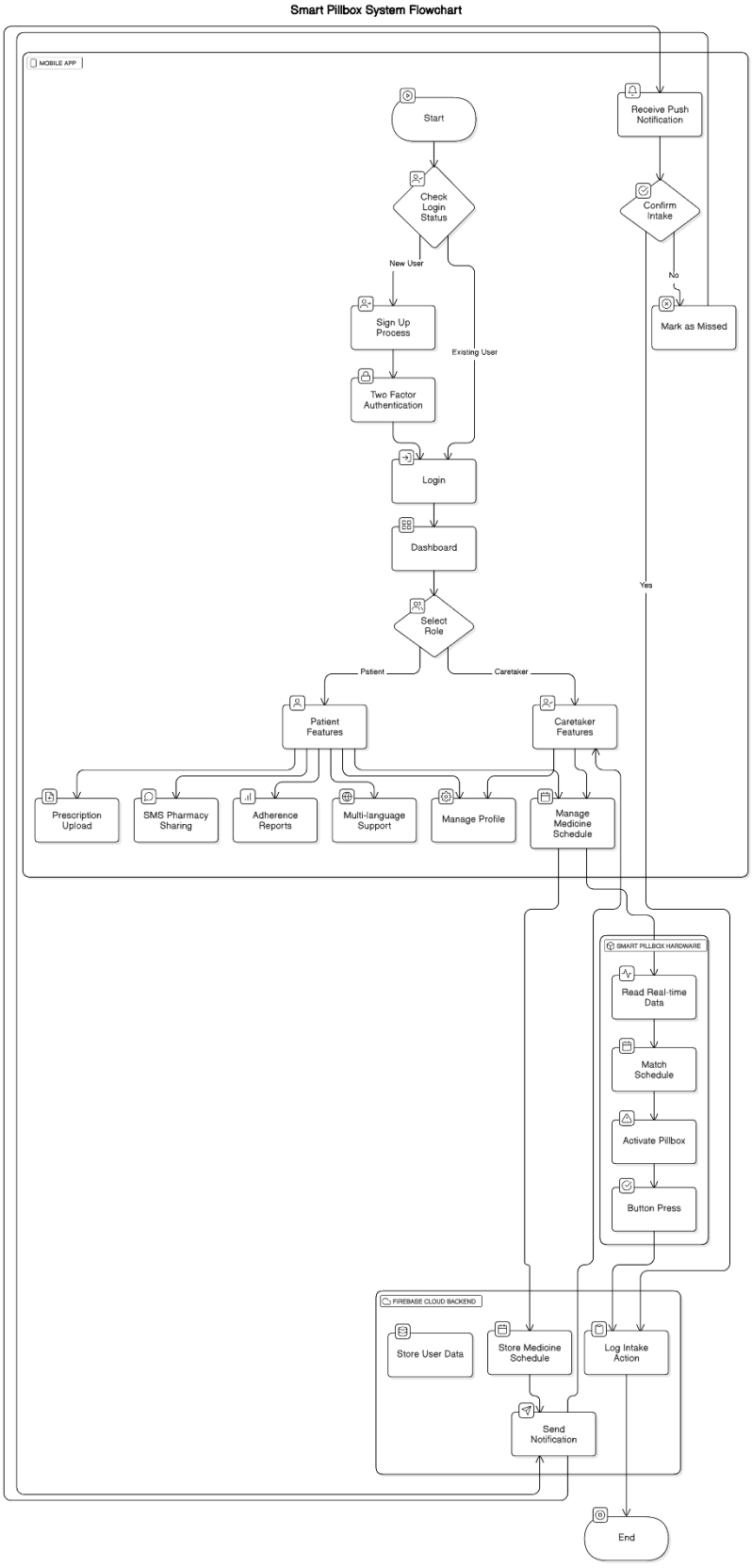


Figure 6.1 System Flowchart

6.2 Reminders Flowchart

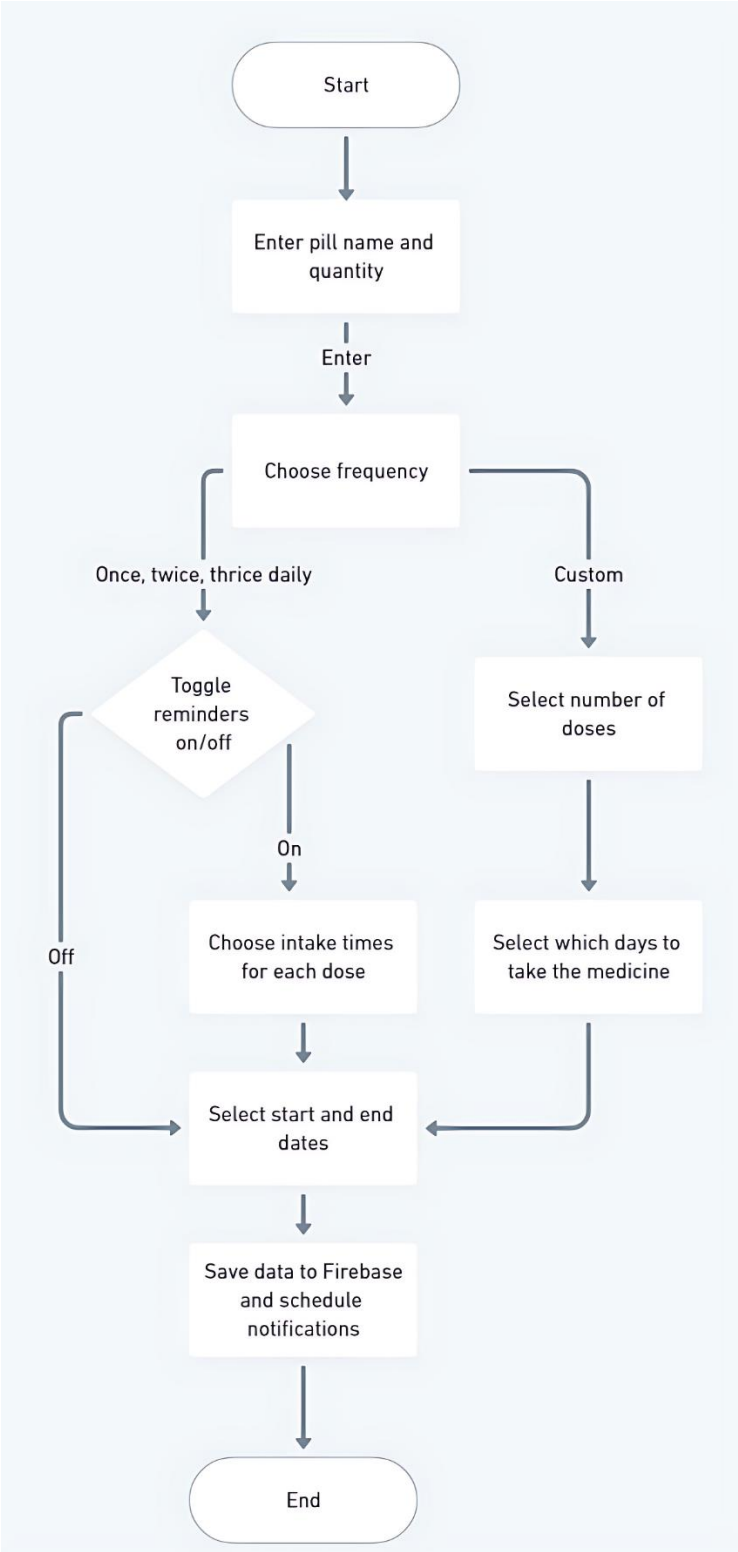


Figure 6.2 Reminder Flowchart

6.3 Caretaker Flowchart

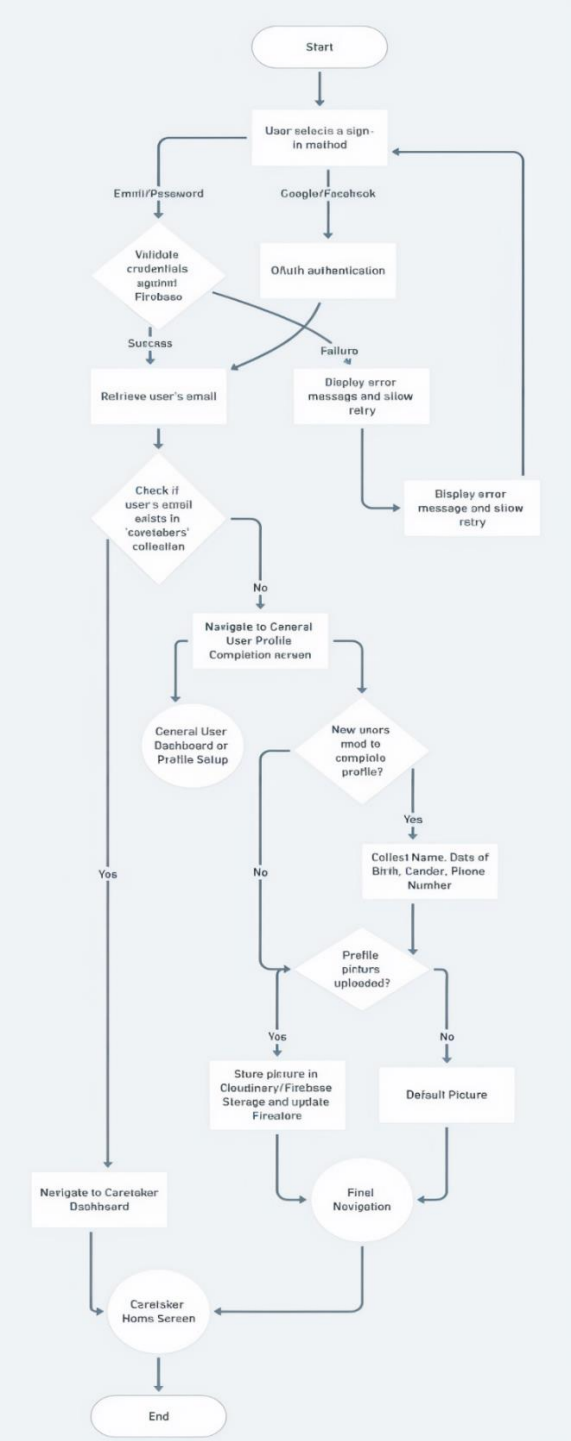


Figure 6.3 Caretaker Flowchart

6.4 AI Assistant

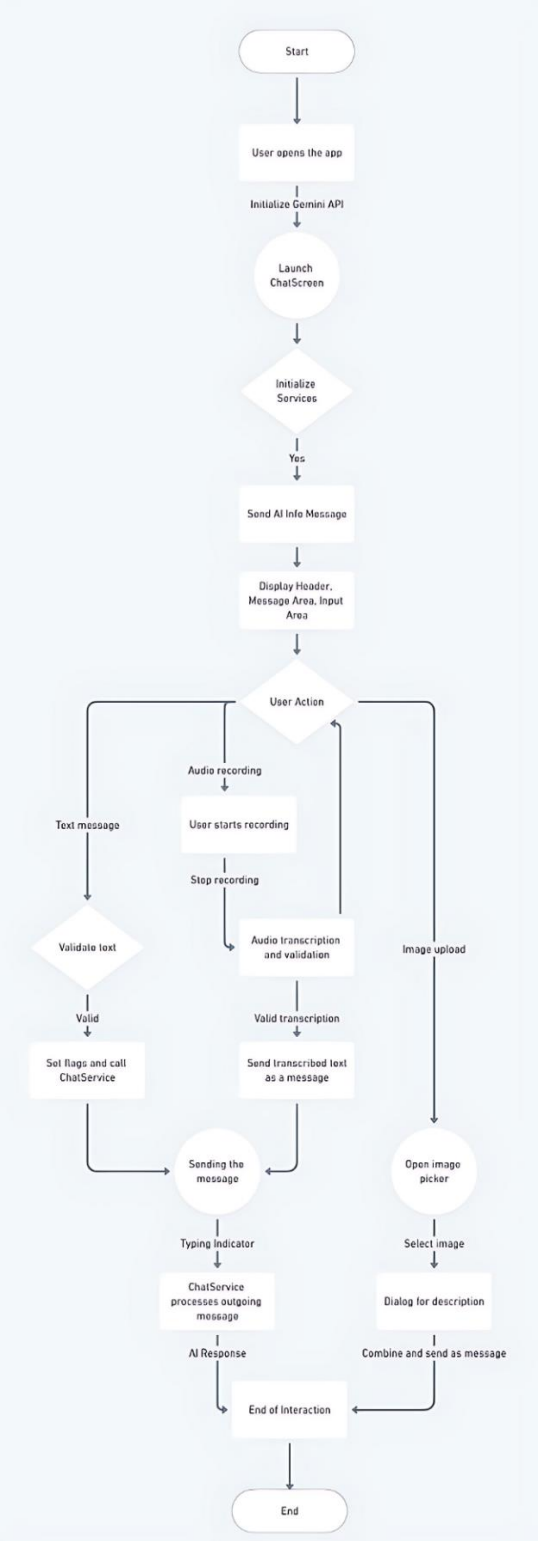


Figure 6.4 AI Assistant Flowchart

6.5 Reports Flowchart

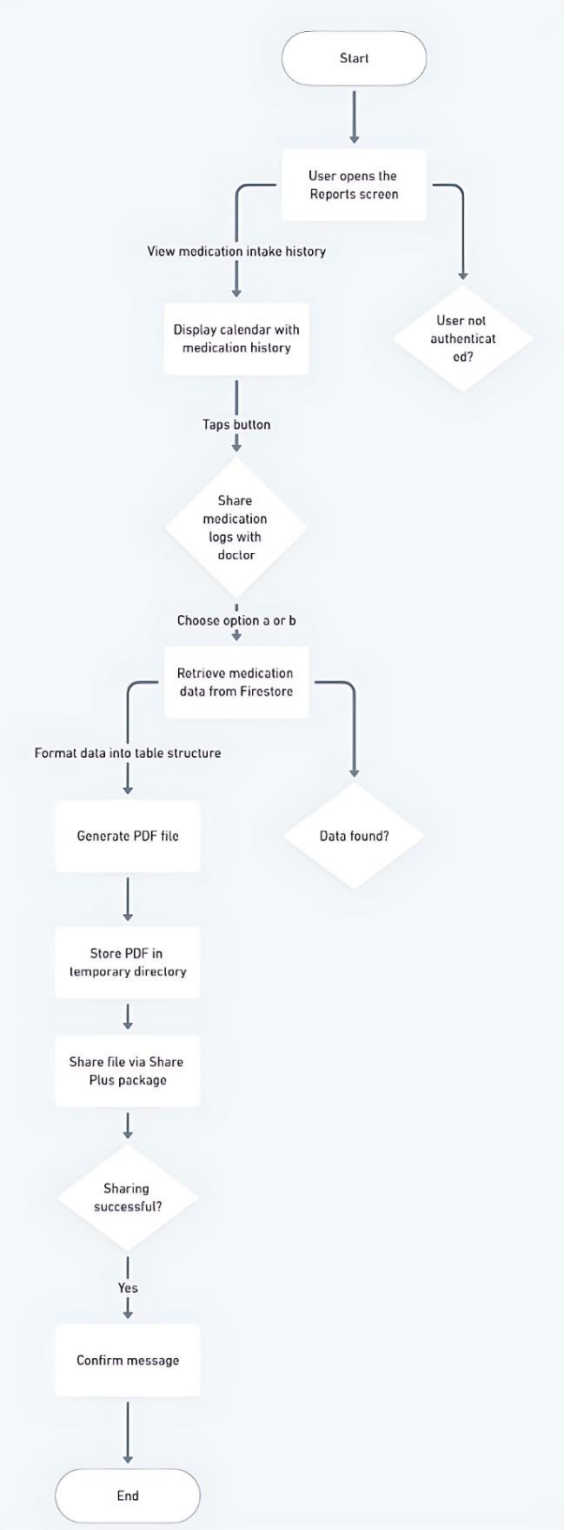


Figure 6.5 Reports Flowchart

6.6 Nearby Pharmacy Flowchart

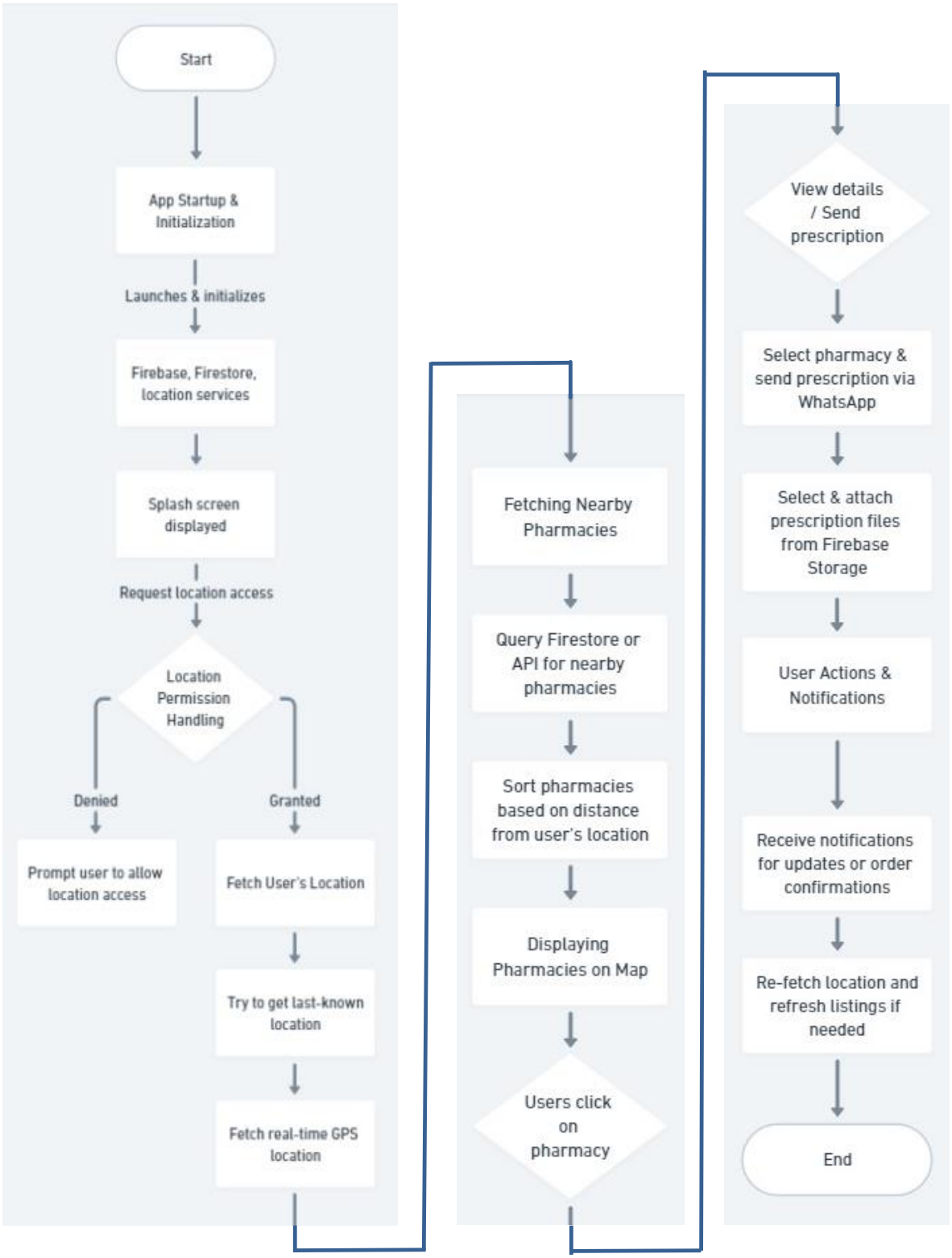


Figure 6.6 Near By Pharmacy Flowchart

6.7 Smart Diagnosis Flowchart

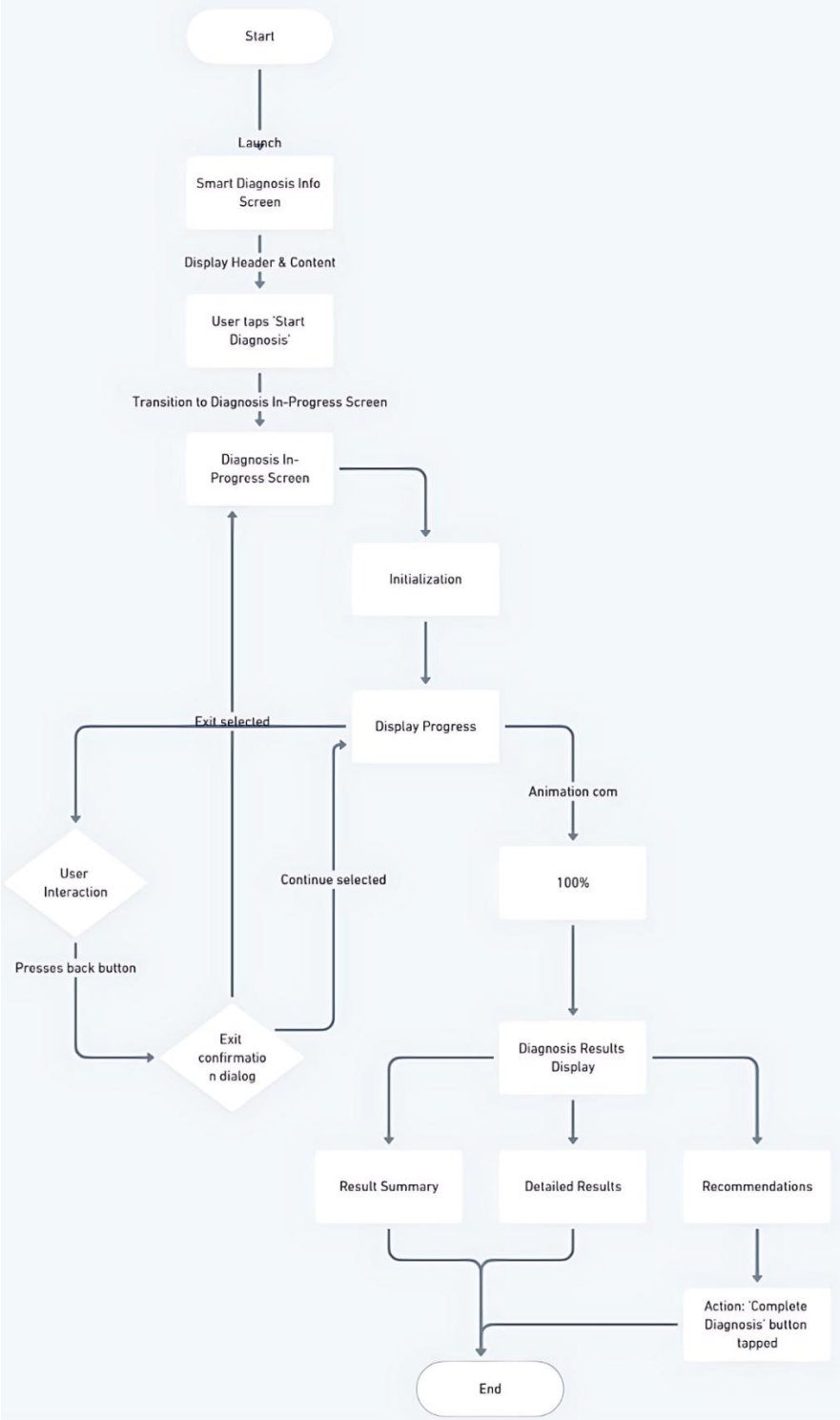


Figure 6.7 Smart Diagnosis Flowchart

6.8 Pill Detection Flowchart

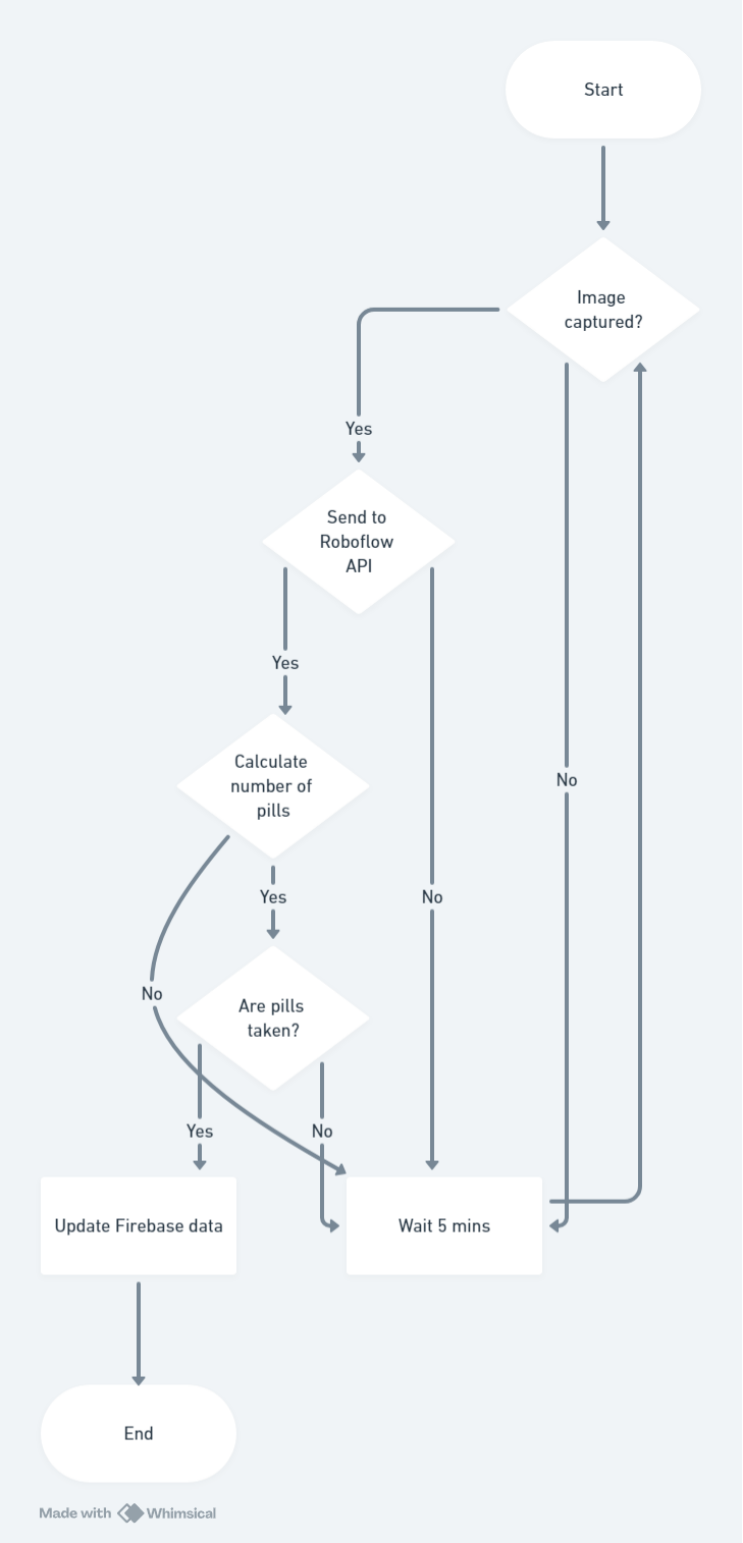


Figure 6.8 Pill Detection Flowchart

6.9 Scan Prescription Flowchart

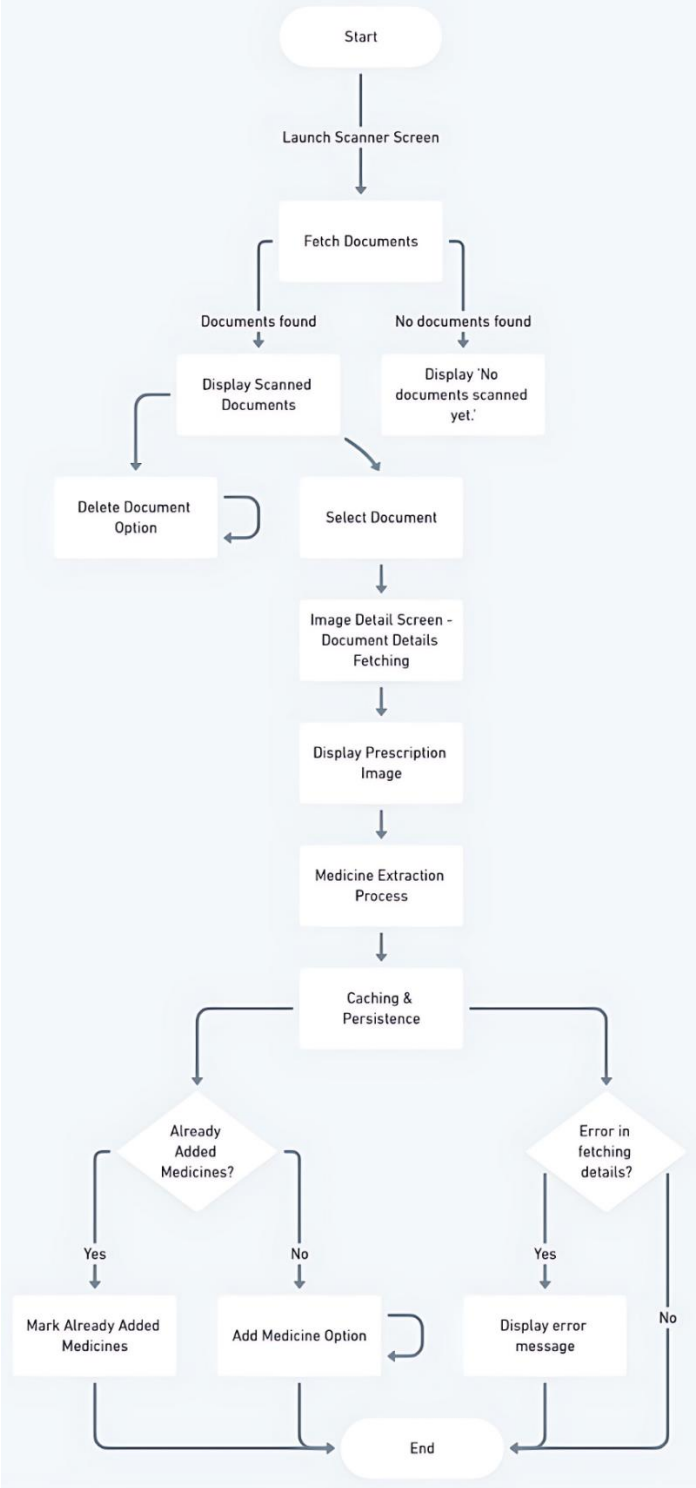


Figure 6.9 Scan Prescription

6.10 Scan Medicine Flowchart

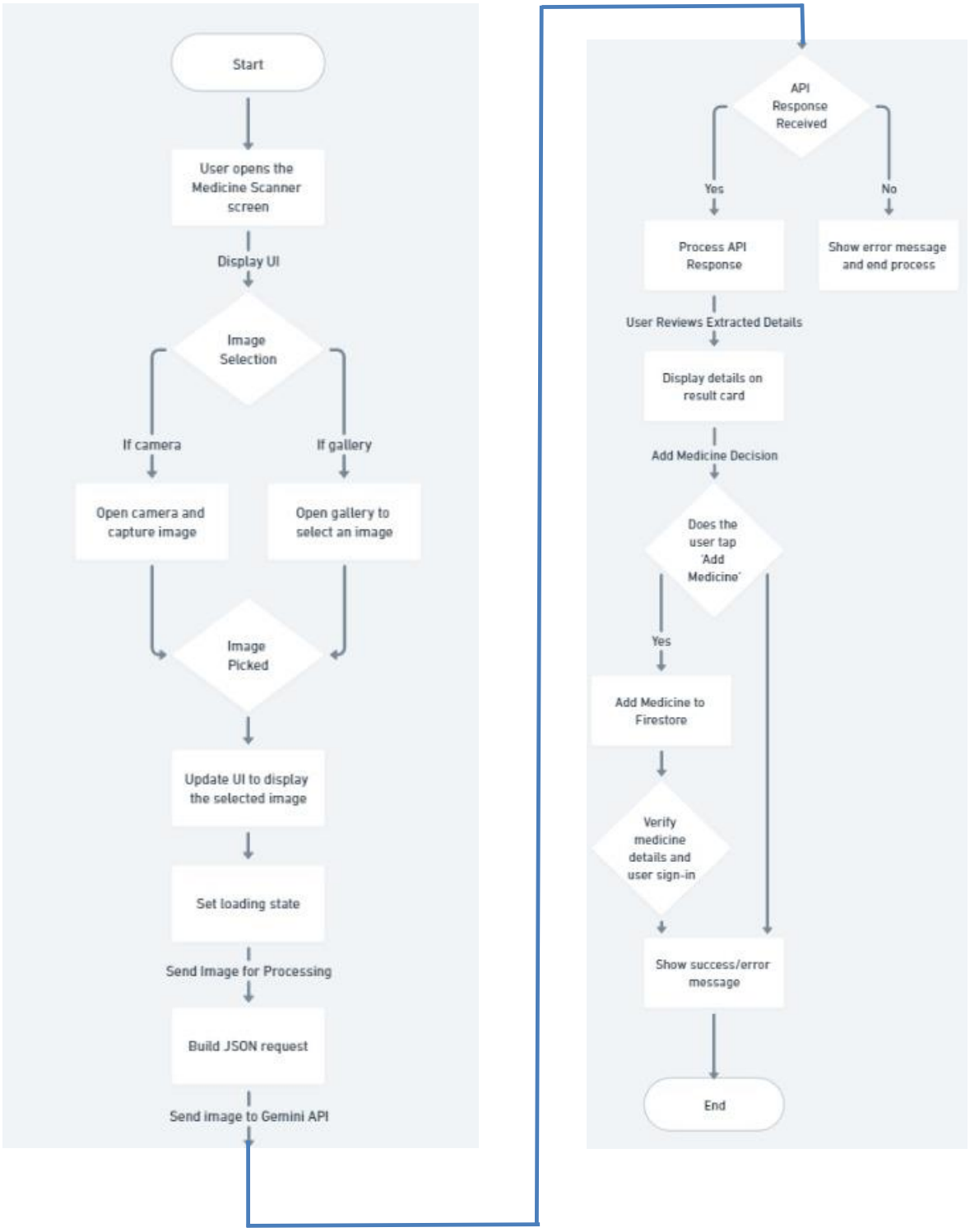


Figure 6.10 Scan Medicine Flowchart

6.11 Smart Config Flowchart

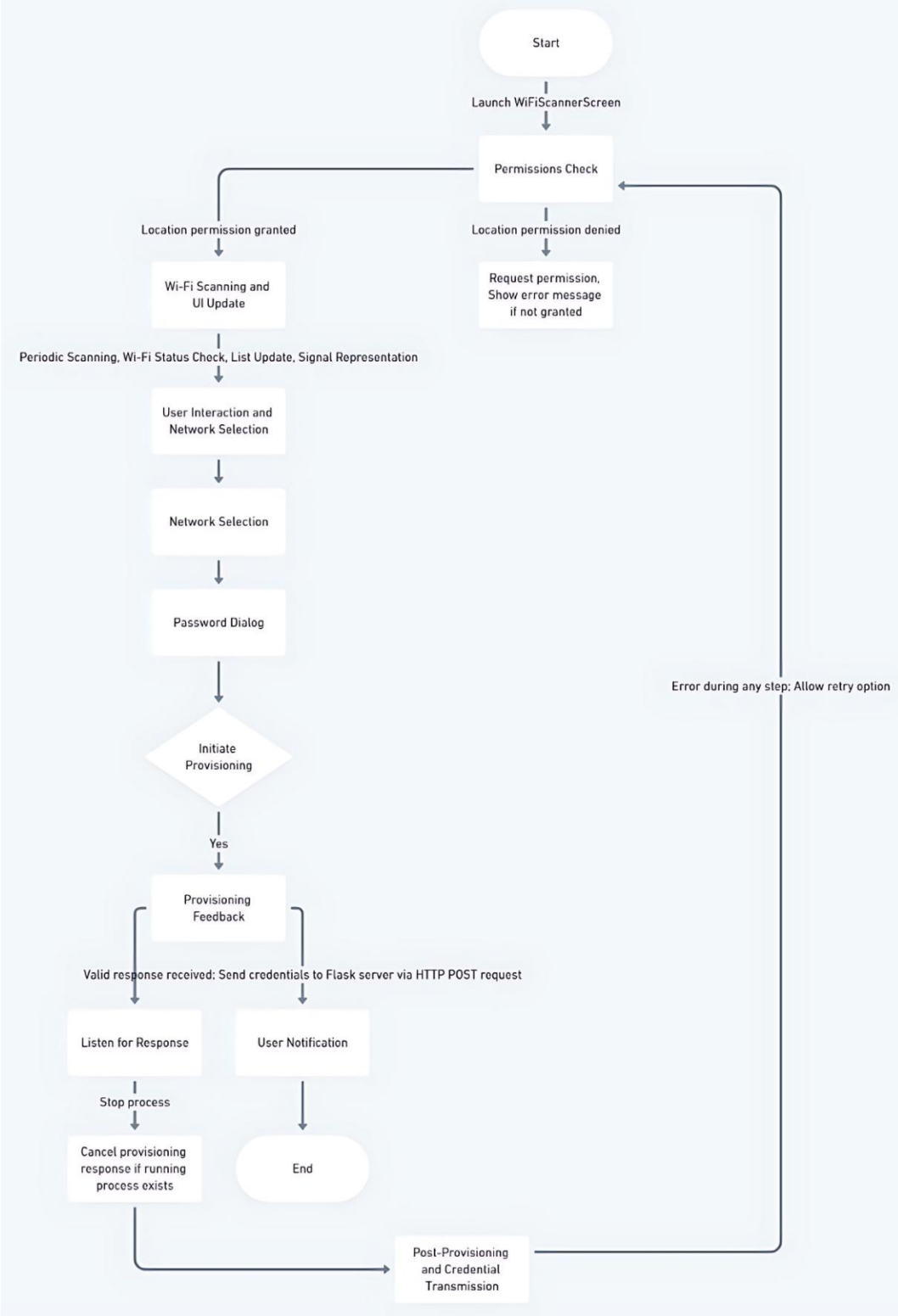


Figure 6.11 Smart Config Flowchart

6.12 Use Case Diagram

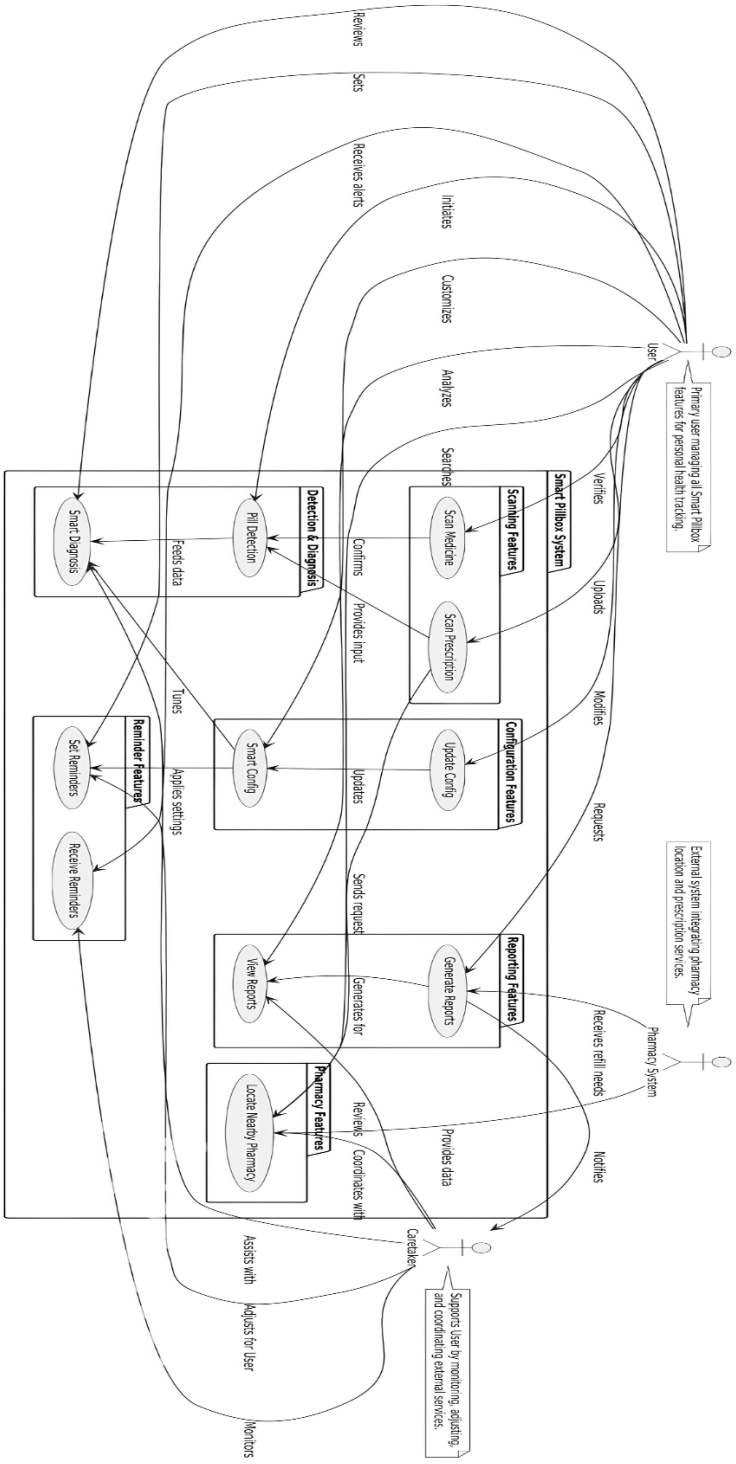


Figure 6.12 Use Case Diagram

6.13 Gantt Chart

ID	Task Name	Start	End	Duration
1	Requirements and Specifications	02-01-2025	09-01-2025	8 days
2	Hardware Development and Integration	10-01-2025	25-01-2025	16 days
3	Image Processing and Pill Detection Software	26-01-2025	23-02-2025	29 days
4	System Integration and Communication	24-02-2025	17-03-2025	22 days
5	User Interface and Experience	18-03-2025	08-04-2025	22 days
6	Testing and Bug Fixing	09-04-2025	21-04-2025	13 days

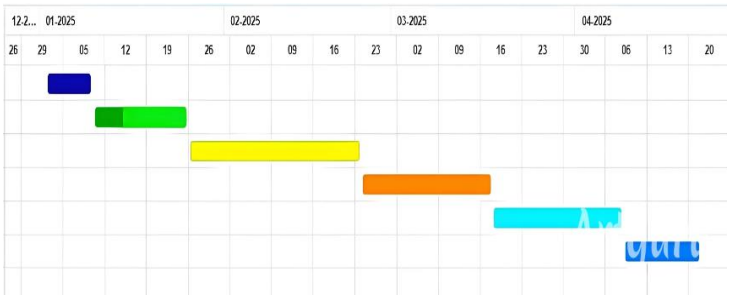


Figure 6.13 Gantt Chart

Chapter 7

Implementation

7.1 Technology Review:

7.1.1 Flutter:

Flutter is an open-source UI software development framework created by Google, used to develop cross-platform applications for Android, iOS, web, and desktop from a single codebase. It uses the Dart programming language and offers a fast development cycle with features like hot reload, a rich widget library, and expressive UI. Flutter is especially well-suited for building highly interactive and visually appealing mobile apps like the Smart PillBox companion app.



Figure 7.1.1 Flutter

7.1.2 Kotlin:

Kotlin is a statically typed programming language developed by JetBrains and officially supported for Android development. It's concise, expressive, and fully interoperable with Java. Kotlin is often used in Android systems for managing alarms and notifications due to its modern syntax and improved type safety, making it suitable for implementing the pill reminder and alarm system in your app.



Figure 7.1.2 Kotlin

7.1.3 Firebase:

Firebase is a cloud-based platform by Google that provides a suite of tools including real-time databases, authentication, cloud messaging, and analytics. For the Smart PillBox, Firebase handles real-time data syncing between the mobile app and the ESP32 module, including user data, pill schedules, scanned prescription data, and notifications. It eliminates the need for backend server setup, making development faster and more scalable.

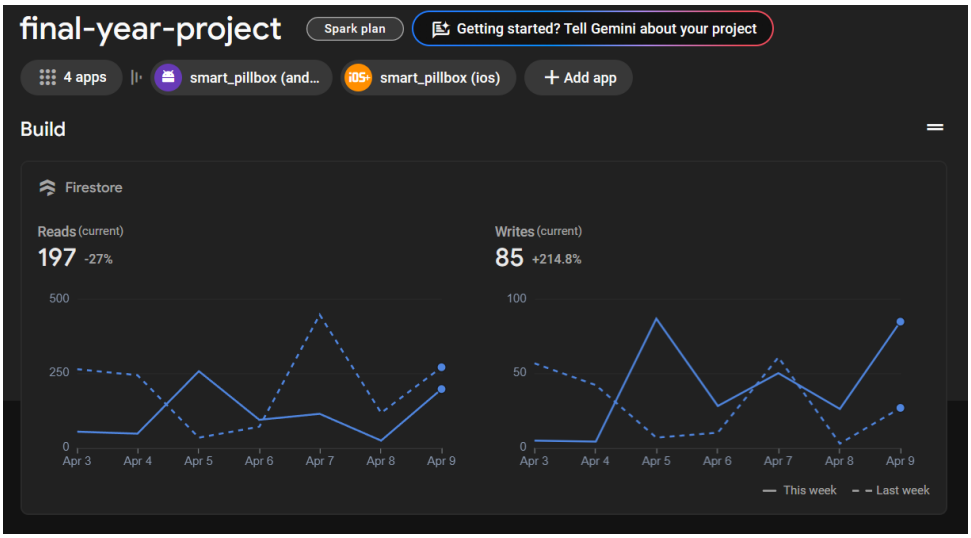


Figure 7.1.3 Firebase

7.1.4 Python:

Python is a high-level, interpreted programming language known for its readability and extensive libraries. It is widely used for backend development, machine learning, and API creation. In the Smart PillBox system, Python is used to develop APIs that interact with external services like Gemini for scanning prescriptions and medicine labels. Frameworks like Flask or FastAPI make Python ideal for creating lightweight, efficient APIs.



Figure 7.1.4 Python

7.1.5 C++:

C++ is a powerful, low-level programming language commonly used in embedded systems and microcontroller programming. In the Smart PillBox project, C++ is used for programming the ESP32 microcontroller, handling operations like motor control, sensor data reading, Wi-Fi connectivity, and interaction with peripherals like the display, buzzer, and camera. Its efficiency and hardware-level control make it ideal for real-time embedded systems.



Figure 7.1.5 C++

7.2 User Interface:

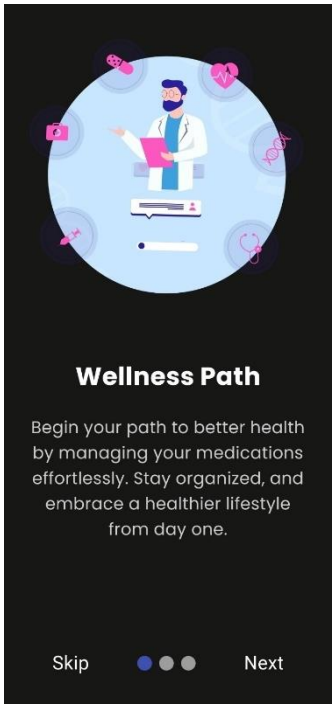


Figure 7.2.1
Onboarding Screen 1

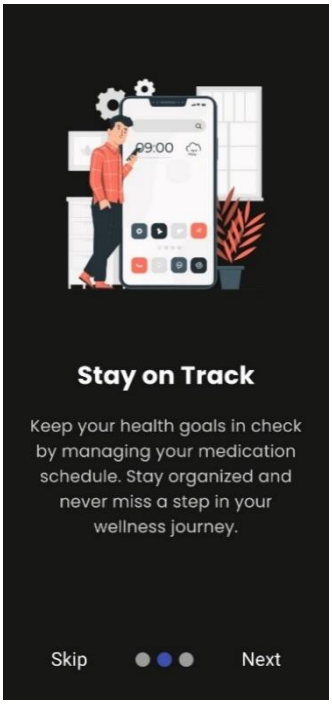


Figure 7.2.2
Onboarding Screen 2



Figure 7.2.3
Onboarding Screen 3

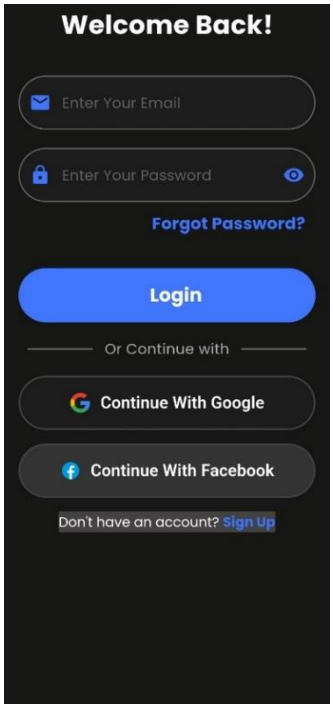


Figure 7.2.4
Login Screen

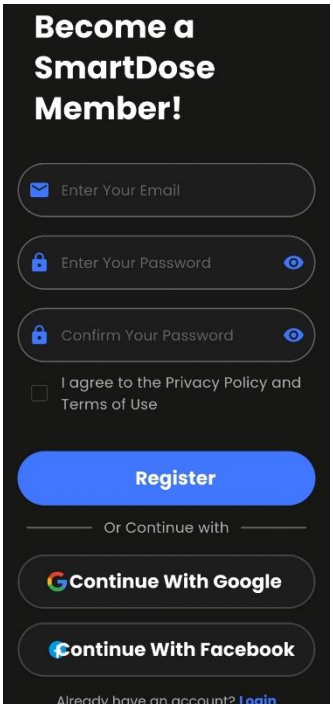


Figure 7.2.5
Sign Up Screen

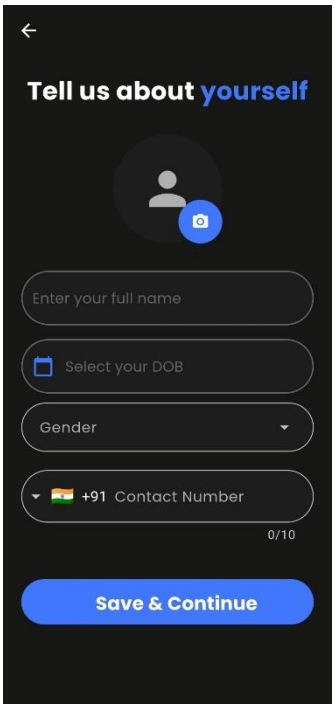


Figure 7.2.6
Registration Screen

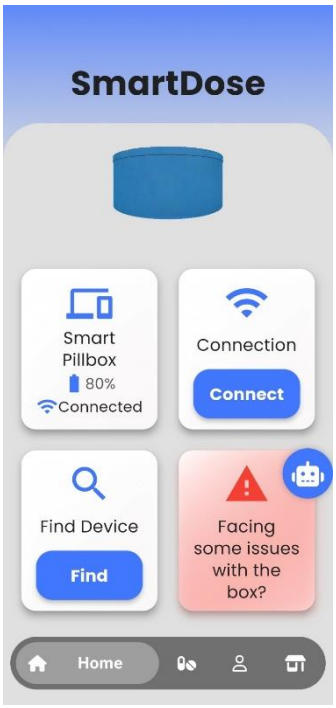


Figure 7.2.7
Home Screen



Figure 7.2.8
Connection Screen

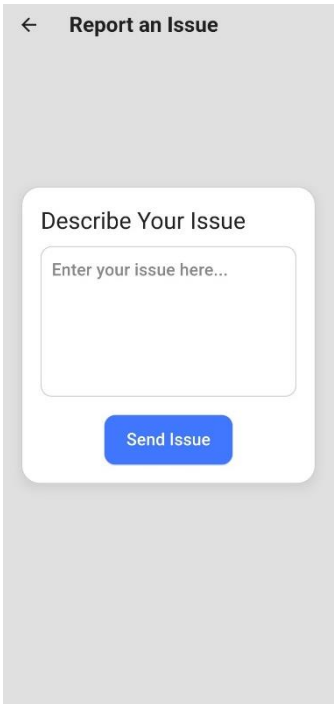


Figure 7.2.9
Issue Screen

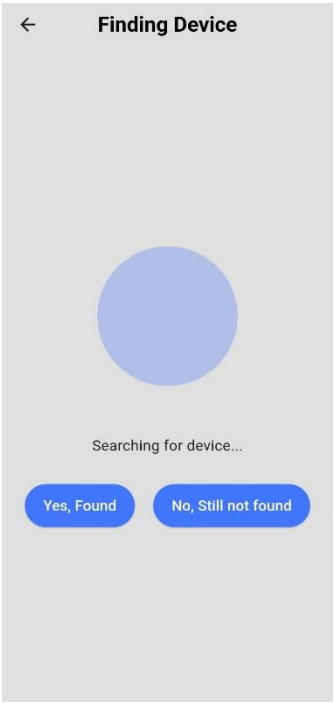


Figure 7.2.10
Find Device Screen

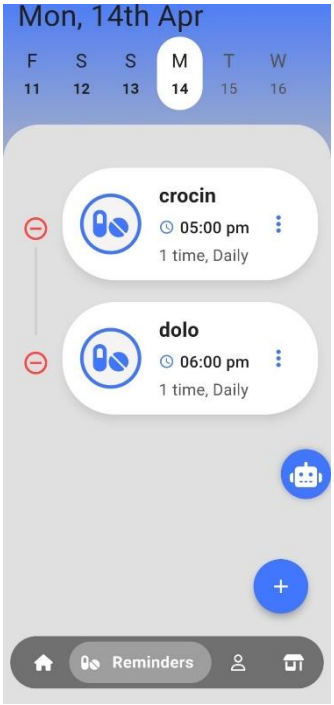


Figure 7.2.11
Medicine List Screen

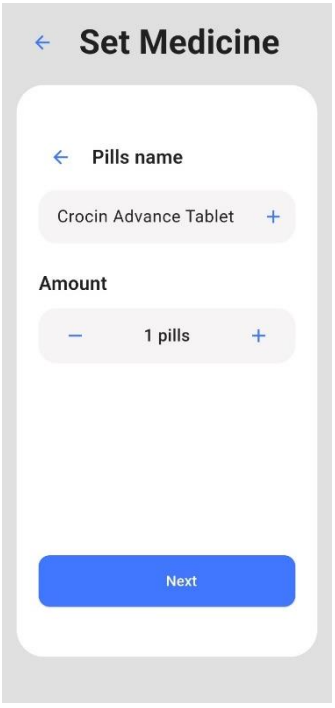


Figure 7.2.12
Add Medicine Screen

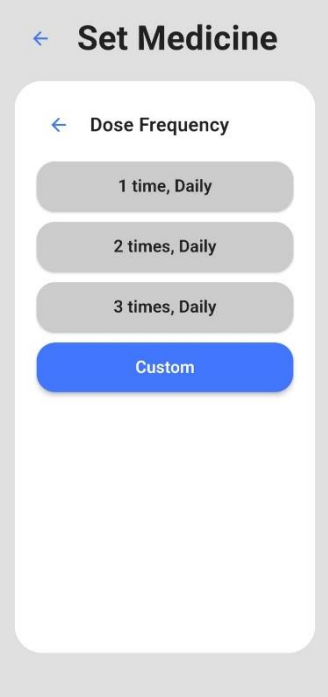


Figure 7.2.13
Schedule Medicine Screen 1

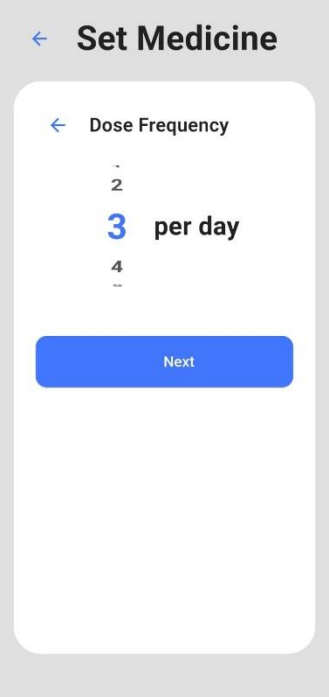


Figure 7.2.14
Schedule Medicine Screen 2

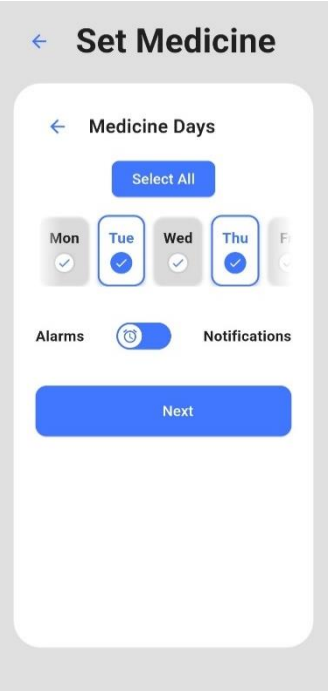


Figure 7.2.15
Schedule Medicine Screen 3

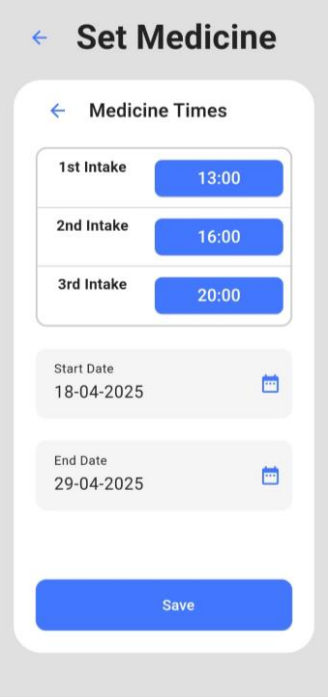


Figure 7.2.16
Schedule Medicine Screen 4

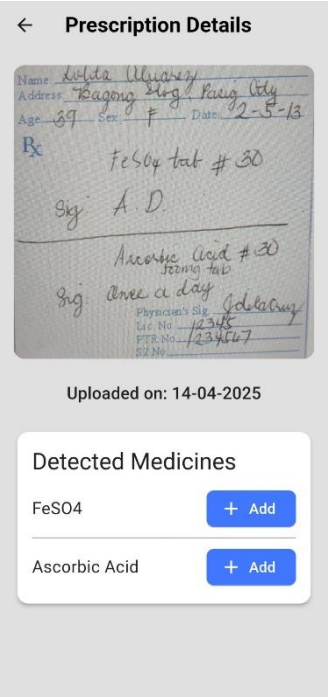


Figure 7.2.17
Scan Prescription Screen

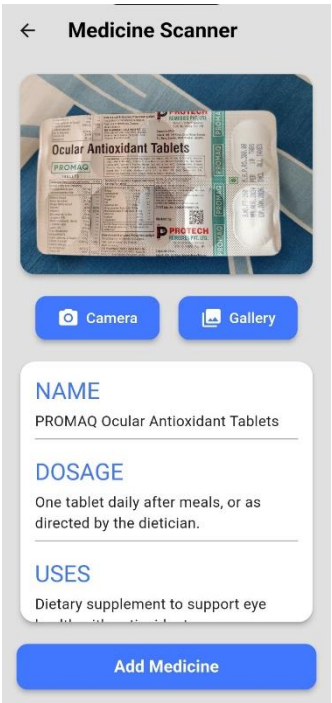


Figure 7.2.18
Scan Medicine Screen

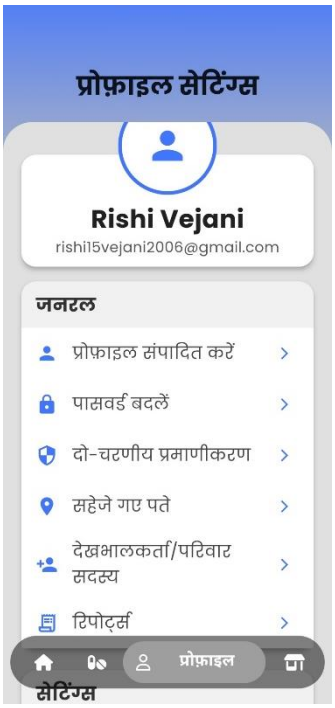


Figure 7.2.19
Profile Screen 1



Figure 7.2.20
Profile Screen 2

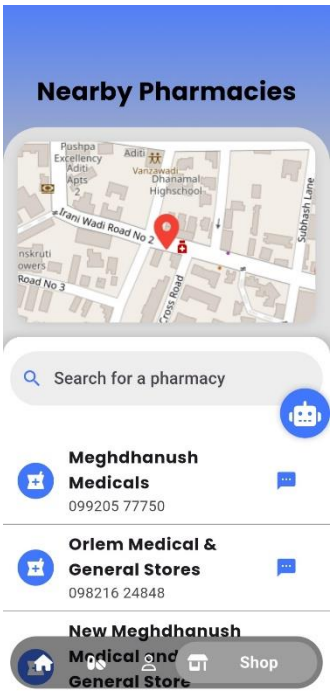


Figure 7.2.21
Pharmacy Home Screen

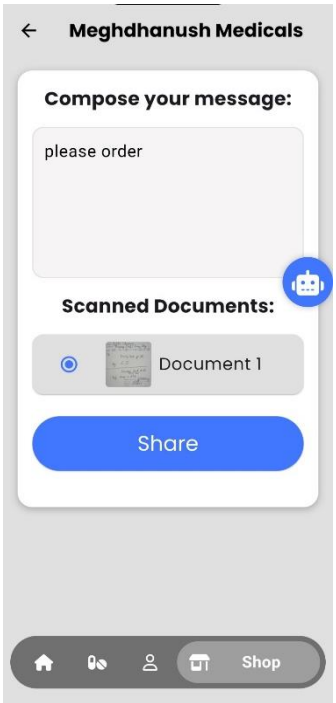


Figure 7.2.22
Pharmacy Message Screen

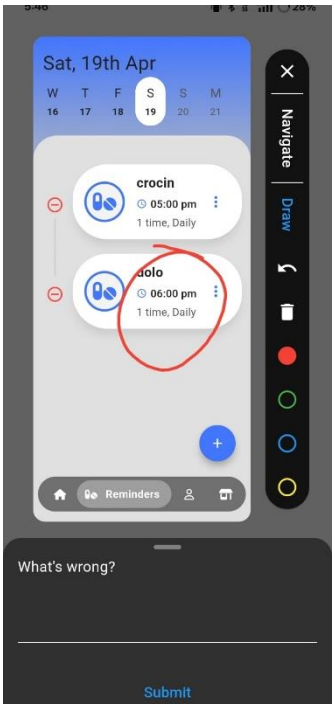


Figure 7.2.23
Shake to Report 1

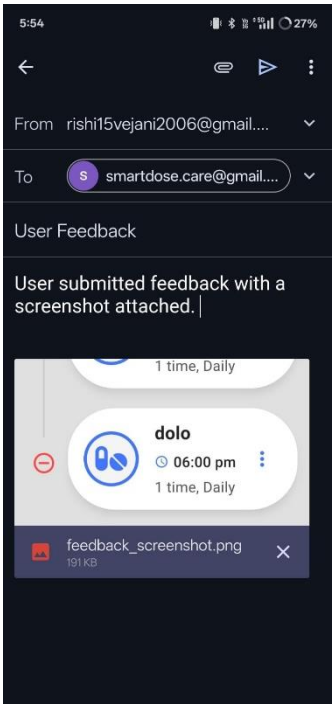


Figure 7.2.24
Shake to Report 2

7.3 Software Code:

7.3.1 main.dart:

```
final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();
Position? userLocation; List<dynamic>? pharmacyData; MapController mapController = MapController(
  initPosition: GeoPoint(latitude: 0, longitude: 0),
);

void callMedicineCheck() async {
    User? user = FirebaseAuth.instance.currentUser;

  if (user != null) {
    String userId = user.uid;     bool isNotification = false;     debugPrint(userId);

    checkMedicineTimes(userId, isNotification);
  } else {
    print("No user is signed in.");
  }
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await ThemeProvider.init();
  await EasyLocalization.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);

    const apiKey = 'AIzaSyD3psw8M8hiX2mnwGoXxc-0-ZvBxPa0IYY';
  Gemini.init(apiKey: apiKey);

    final cron = Cron();
```

```
  cron.schedule(Schedule.parse('0 0 * * *'), () {
    callMedicineCheck();
  });

  await dotenv.load(fileName: "assets/.env");

    runApp(
    EasyLocalization(
      supportedLocales: [
        Locale('en', 'US'),
        Locale('hi', 'IN'),
        Locale('gu', 'IN'),          Locale('mr', 'IN'),        ],
      path: 'assets/translations',        fallbackLocale: Locale('en', 'US'),
      child: const BetterFeedback(
        child: ShakeFeedbackWrapper(
          child: MyApp(),
        ),
      ),
    ),
  );
}

class AuthWrapper extends StatelessWidget {
  const AuthWrapper({super.key});

  Future<bool> isCaretaker(String email) async {
    final querySnapshot = await firestore.FirebaseFirestore.instance
        .collection('caretakers')
        .where('email', isEqualTo: email)
        .get();
    return querySnapshot.docs.isNotEmpty;
```

```
}

@override
Widget build(BuildContext context) {
    requestPermissions(context);

 return StreamBuilder<User?>(
   stream: FirebaseAuth.instance.authStateChanges(),
   builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.active) {
      if (snapshot.hasData) {
        final user = snapshot.data!;
        return FutureBuilder<bool>(
          future: isCaretaker(user.email!),
          builder: (context, caretakerSnapshot) {
                    Widget homeScreen = caretakerSnapshot.data == true
              ? const CareTakerHomeScreen()
              : const HomeScreen();
                    return FloatingDraggableWidget(
            dx: MediaQuery.of(context).size.width -
              55,                 dy: MediaQuery.of(context).size.height *
              0.65,                 mainScreenWidget: homeScreen,
            floatingWidget: GestureDetector(
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                      builder: (context) => const ChatScreen()),
                );
              },
              child: Container(
```

```
                width: 60,
                height: 60,
                decoration: BoxDecoration(
                  color: AppColors.buttonColor,
                  shape: BoxShape.circle,
                  boxShadow: [
                    BoxShadow(color: Colors.black26, blurRadius: 5),
                  ],
                ),
                child: const Icon(FontAwesomeIcons.robot,
                    color: Colors.white, size: 28),
              ),
            ),
            autoAlign:
                false,              floatingWidgetWidth: 55,
            floatingWidgetHeight: 55,
            speed: 0.5,
          );
        },
      );
    } else {
            return LanguageSelectionScreen();
    }
  } else {
    return const Center(child: CircularProgressIndicator());
  }
 },
);
}

Future<void> saveTokenToFirestore(String token) async {
```

```
try {
  final user = FirebaseAuth.instance.currentUser;
  final prefs = await SharedPreferences.getInstance();

  if (user != null) {
    final userId = user.uid;
    final normalizedEmail = user.email?.trim().toLowerCase() ?? '';

        final caretakerQuery = await firestore.FirebaseFirestore.instance
        .collection('caretakers')
        .where('email', isEqualTo: normalizedEmail)
        .get();

    if (caretakerQuery.docs.isNotEmpty) {
      final caretakerDocId = caretakerQuery.docs.first.id;
      await firestore.FirebaseFirestore.instance
          .collection('caretakers')
          .doc(caretakerDocId)
          .set({
        'deviceToken': token,
        'name': user.displayName ?? '',
        'email': user.email ?? '',
      }, firestore.SetOptions(merge: true));
      print("Caretaker token updated for: $normalizedEmail");
    } else {
      await firestore.FirebaseFirestore.instance
          .collection('users')
          .doc(userId)
          .set({
        'deviceToken': token,
      }, firestore.SetOptions(merge: true));
```

```
        print("User token updated for: ${user.email}");
      }
    } else {
      await prefs.setString('fcm_token', token);
      print("Token stored locally in SharedPreferences");
    }
  } catch (e) {
    print('Error saving token: $e');
  }
}

Future<String> writeScreenshotToFile(Uint8List screenshotBytes) async {
  final directory = await getTemporaryDirectory();
  final filePath = '${directory.path}/feedback_screenshot.png';
  final file = File(filePath);
  await file.writeAsBytes(screenshotBytes);
  return filePath;
}
```

7.3.2 routes.dart:

```
class Routes {
  static const String splash = '/';
  static const String onboarding = '/onboarding';
  static const String home = '/home';
  static const String caretakerHome = '/caretakerHome';
  static const String login = '/login';
  static const String signup = '/signup';
  static const String forgotPassword = '/forgotpassword';
  static const String userProfile = '/userProfile';
  static const String editProfile = '/editProfile';
```

```
static const String userinfoscreen = '/userinfoscreen';
static const String profileCompletion = '/profileCompletion';

static Route<dynamic> generateRoute(RouteSettings settings) {
  switch (settings.name) {
    case splash:
      return MaterialPageRoute(builder: (_) => const SplashScreen());

    case onboarding:
      return MaterialPageRoute(builder: (_) => const OnboardingScreen());

    case login:
      return MaterialPageRoute(builder: (_) => const LoginScreen());

    case signup:
      return MaterialPageRoute(builder: (_) => const SignupScreen());

    case forgotPassword:
      return MaterialPageRoute(builder: (_) => const ForgotPasswordScreen());

    case userinfoscreen:
      return MaterialPageRoute(builder: (_) => const UserInfoScreen());

    case profileCompletion:
      return MaterialPageRoute(
          builder: (_) => const ProfileCompletionScreen());

    case home:
      return MaterialPageRoute(builder: (_) => const HomeScreen());

    case caretakerHome:
```

```
      return MaterialPageRoute(builder: (_) => const CareTakerHomeScreen());

    case userProfile:
      return MaterialPageRoute(builder: (_) => const UserProfileScreen());

    default:
      return MaterialPageRoute(
        builder: (_) => Scaffold(
          body: Center(child: Text('No route defined for ${settings.name}')),
        ),
      );
  }
 }
}
```

7.3.3 firebase_options.dart:

```
class DefaultFirebaseOptions {
 static FirebaseOptions get currentPlatform {
  if (kIsWeb) {
   return web;
  }
  switch (defaultTargetPlatform) {
   case TargetPlatform.android:
    return android;
   case TargetPlatform.iOS:
    return ios;
   case TargetPlatform.macOS:
    return macos;
   case TargetPlatform.windows:
    return windows;
```

```
    case TargetPlatform.linux:
      throw UnsupportedError(
        'DefaultFirebaseOptions have not been configured for linux - '
        'you can reconfigure this by running the FlutterFire CLI again.',
      );
    default:
      throw UnsupportedError(
        'DefaultFirebaseOptions are not supported for this platform.',
      );
  }
}

static const FirebaseOptions web = FirebaseOptions(
  apiKey: 'AIzaSyB3SI_n2zLPLil4gjh91W5Ms1HprpGdDVo',
  appId: '1:353991824699:web:2a7162b54a6117d9ba931e',
  messagingSenderId: '353991824699',
  projectId: 'final-year-project-6ebae',
  authDomain: 'final-year-project-6ebae.firebaseapp.com',
  storageBucket: 'final-year-project-6ebae.appspot.com',
);

static const FirebaseOptions android = FirebaseOptions(
  apiKey: 'AIzaSyB5VZ-KQtlO_KzGwdV0DFHlYfULBKIuVkI',
  appId: '1:353991824699:android:ac89eefd45282a4eba931e',
  messagingSenderId: '353991824699',
  projectId: 'final-year-project-6ebae',
  storageBucket: 'final-year-project-6ebae.appspot.com',
);

static const FirebaseOptions ios = FirebaseOptions(
  apiKey: 'AIzaSyBDONImHV_jh3bQU4Fs-v0oy0LxPJ6nwGI',
```

```
    appId: '1:353991824699:ios:b51cf71369996e5cba931e',
    messagingSenderId: '353991824699',
    projectId: 'final-year-project-6ebae',
    storageBucket: 'final-year-project-6ebae.appspot.com',
    iosClientId: '353991824699-
hv9md6phhmkuprgomhog44m174tbjdvr.apps.googleusercontent.com',
    iosBundleId: 'com.example.smart_pillbox',
  );

  static const FirebaseOptions macos = FirebaseOptions(
    apiKey: 'AIzaSyBDONImHV_jh3bQU4Fs-v0oy0LxPJ6nwGI',
    appId: '1:353991824699:ios:b51cf71369996e5cba931e',
    messagingSenderId: '353991824699',
    projectId: 'final-year-project-6ebae',
    storageBucket: 'final-year-project-6ebae.appspot.com',
    iosClientId:'353991824699-
hv9md6phhmkuprgomhog44m174tbjdvr.apps.googleusercontent.com',
    iosBundleId: 'com.example.smart_pillbox',
  );
  static const FirebaseOptions windows = FirebaseOptions(
    apiKey: 'AIzaSyB3SI_n2zLPLil4gjh91W5Ms1HprpGdDVo',
    appId: '1:353991824699:web:ec914a2b71297685ba931e',
    messagingSenderId: '353991824699',
    projectId: 'final-year-project-6ebae',
    authDomain: 'final-year-project-6ebae.firebaseapp.com',
    storageBucket: 'final-year-project-6ebae.appspot.com',
  );
}
```

7.3.4 chat_screen.dart (AI):

```
class ChatScreen extends StatefulWidget {
  const ChatScreen({super.key});

  @override
  State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
  final ChatService _chatService = ChatService();
  final AudioService _audioService = AudioService();

  List<ChatMessage> _messages = [];
  bool _isRecording = false;
  bool _isTyping = false;
  bool _isGeneratingResponse = false;

  @override
  void initState() {
    super.initState();
    _initServices();
  }

  Future<void> _initServices() async {
    await _chatService.init();
    await _audioService.initialize();
    await Future.delayed(const Duration(milliseconds: 100));
    _sendAiInfoMessage();
    setState(() {
      _messages = _chatService.messages;
```

```
    });
  }

  void _sendAiInfoMessage() {
    final aiMessage = ChatMessage(
      isUser: false,
      text:
          "Hello! I'm SmartDose, your health assistant. How can I help you today with your health-related questions?"
              .tr(),
    );
    _chatService.messages.add(aiMessage);


    setState(() {
      _messages = _chatService.messages;
    });
  }
  void _sendMessage({String? text, File? file}) async {
    if ((text == null || text.trim().isEmpty) && file == null) return;

    setState(() {
      _isTyping = true;
      _isGeneratingResponse = true;
    });
    await _chatService.sendMessage(text: text, file: file);
    setState(() {
      _messages = _chatService.messages;
      _isTyping = false;
      _isGeneratingResponse = false;
    });
```

```
}
Future<void> _uploadImage() async {
  final ImagePicker picker = ImagePicker();
  final XFile? pickedFile =
      await picker.pickImage(source: ImageSource.gallery);

  if (pickedFile != null) {
    File image = File(pickedFile.path);
    String description = await _showDescriptionDialog();
    _sendMessage(text: description, file: image);
  }
}
Future<void> _startRecording() async {
  if (!_isGeneratingResponse && !_isRecording) {
    await _audioService.startRecording();
    setState(() => _isRecording = true);
  }
}

Future<void> _stopRecording() async {
  if (_isRecording) {
    String transcribedText = await _audioService.stopRecording();
    setState(() => _isRecording = false);
    _sendMessage(
        text: transcribedText.isNotEmpty
            ? transcribedText
            : 'Failed to transcribe speech');
  }
}

void _stopResponseGeneration() {
```

```
    setState(() {
      _isGeneratingResponse = false;
      _isTyping = false;
    });
  }
}
```

7.3.5 login_screen.dart:

```
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.background, // Theme-aware background
      resizeToAvoidBottomInset: true,
      body: const Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Expanded(
            child: LoginForm(),
          ),
        ],
      ),
    );
  }
}
```

7.3.6 signup_screen.dart:

```
class SignupScreen extends StatelessWidget {
  const SignupScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.background, // Theme-aware background
      resizeToAvoidBottomInset: true,
      body: const Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Expanded(
            child: SignupForm(),
          ),
        ],
      ),
    );
  }
}
```

7.3.7 pharmacy_screen.dart:

```
class PharmacyScreen extends StatefulWidget {
  const PharmacyScreen({super.key});

  @override
  _PharmacyScreenState createState() => _PharmacyScreenState();
}
class _PharmacyScreenState extends State<PharmacyScreen> {
  bool _dataFetched = false;
```

```
  @override
  void initState() {
    super.initState();
    _waitForData();
  }
  Future<void> _waitForData() async {
    while (pharmacyData == null || mapController == null) {
      await Future.delayed(const Duration(milliseconds: 100));
    }
    await Future.delayed(const Duration(seconds: 1));
    setState(() {
      _dataFetched = true;
    });
  }
}
```

7.3.8 medicine_list_screen.dart:

```
class MedicineListScreen extends StatefulWidget {
  const MedicineListScreen({super.key});
  @override
  State<MedicineListScreen> createState() => _MedicineListScreenState();
}
class _MedicineListScreenState extends State<MedicineListScreen> {
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;
  final GlobalKey<ScaffoldMessengerState> scaffoldMessengerKey =
      GlobalKey<ScaffoldMessengerState>();
  final MedicineFormScreenState medicineFormScreenState =
      MedicineFormScreenState();
  late DateTime today;
  late DateTime selectedDate;
```

```
  late ScrollController _scrollController;
  int _selectedIndex = 0;

  late ValueNotifier<bool> _isSpeedDialOpen;
  @override
  void initState() {
    super.initState();
    today = DateTime.now();
    selectedDate = today;
    _scrollController = ScrollController();
    _isSpeedDialOpen = ValueNotifier<bool>(false);
    WidgetsBinding.instance.addPostFrameCallback((_) {
      _scrollToDate();
    });
  }
  @override
  void dispose() {
    _scrollController.dispose();
    _isSpeedDialOpen.dispose();
    super.dispose();
  }
  void _scrollToDate() {
    scrollToDate(context, _scrollController, today, selectedDate);
  }
}
```

7.3.9 scanner_screen.dart:

```
  class ScannerScreen extends StatefulWidget {
    final String userId;
    const ScannerScreen({super.key, required this.userId});
```

```
  @override
  _ScannerScreenState createState() => _ScannerScreenState();
 }
 class _ScannerScreenState extends State<ScannerScreen> {
  final DocumentService _documentService = DocumentService();
  List<Map<String, dynamic>> _scannedDocuments = [];

  @override
  void initState() {
   super.initState();
   _fetchScannedDocuments();
  }
  void _fetchScannedDocuments() async {
   final documents =
       await _documentService.fetchScannedDocuments(widget.userId);
   setState(() {
    _scannedDocuments = documents;
   });
  }
  void _startScanning() async {
   await _documentService.startScanning(widget.userId);
   _fetchScannedDocuments();
  }

  void _deleteDocument(String documentId, String cloudinaryUrl) async {
   await _documentService.deleteDocument(
       widget.userId, documentId, cloudinaryUrl);
   _fetchScannedDocuments();
  }
  }
}
```

7.3.10 MainActivity.kt:

```
class MainActivity : FlutterActivity() {
    private val CHANNEL = "com.example.smart_pillbox/alarm"
    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger,
CHANNEL).setMethodCallHandler { call, result ->
            when (call.method) {
                "scheduleAlarm" -> {
                    val alarmTime = call.argument<Long>("alarmTime")
                    val payload = call.argument<String>("payload")
                    val medicineId = call.argument<String>("medicineId")
                    val selectedDays = call.argument<List<String>>("selectedDays")
                    val endDate = call.argument<Long>("endDate")
                    val startDate = call.argument<Long>("startDate")
                    if (alarmTime != null && payload != null && medicineId != null &&
                        selectedDays != null && endDate != null && startDate != null) {
                        scheduleRecurringAlarm(alarmTime, payload, medicineId, selectedDays,
startDate, endDate)
                        result.success(null)
                    } else {
                        result.error("INVALID_ARGUMENT", "Missing required parameters", null)
                    }
                }
                "cancelAlarm" -> {
                    val medicineId = call.argument<String>("medicineId")
                    if (medicineId != null) {
                        cancelAlarm(medicineId)
                        result.success(null)
                    } else {
                        result.error("INVALID_ARGUMENT", "Medicine ID is null", null)
                    }
```

```
            }
            else -> result.notImplemented()
        }
    }
}
private fun scheduleRecurringAlarm(
    initialAlarmTime: Long,
    payload: String,
    medicineId: String,
    selectedDays: List<String>,
    startDate: Long,
    endDate: Long
) {
    val alarmManager = getSystemService(Context.ALARM_SERVICE) as AlarmManager
    val intent = Intent(this, AlarmReceiver::class.java).apply {
        putExtra("payload", payload)
        putExtra("medicineId", medicineId)
        putExtra("selectedDays", ArrayList(selectedDays))
        putExtra("startDate", startDate)
        putExtra("endDate", endDate)
        putExtra("alarmTime", initialAlarmTime)
    }
    val requestCode = (medicineId + initialAlarmTime.toString()).hashCode()
    val pendingIntent = PendingIntent.getBroadcast(
        this,
        requestCode,
        intent,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
    )
    Log.d("AlarmScheduler", "Scheduling alarm for medicine ID: $medicineId at time: $initialAlarmTime (requestCode: $requestCode)")
```

```
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP,
initialAlarmTime, pendingIntent)
        } else {
            alarmManager.setExact(AlarmManager.RTC_WAKEUP,          initialAlarmTime,
pendingIntent)
        }
        Log.d("AlarmScheduler", "Alarm scheduled successfully for medicine ID: $medicineId at
time: $initialAlarmTime")
    }
    private fun cancelAlarm(medicineId: String) {
        val alarmManager = getSystemService(Context.ALARM_SERVICE) as AlarmManager
        val intent = Intent(this, AlarmReceiver::class.java)
        val pendingIntent = PendingIntent.getBroadcast(
            this,
            medicineId.hashCode(),
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
        alarmManager.cancel(pendingIntent)
        pendingIntent.cancel()
        Log.d("AlarmScheduler", "Alarm canceled for medicine ID: $medicineId")
    }
}
```

7.4 Hardware Code:

7.4.1 webserver.h:

```
#define EEPROM_SIZE 512
WebServer server(80);
```

```
String storedMedicineData = "";
void setupWebServer();
void handleCredentials();
void loadStoredMedicineData();
void saveStoredMedicineData(String data);
void sortMedicineDataAndStore(String rawData);

void loadStoredMedicineData() {
  EEPROM.begin(EEPROM_SIZE);
  storedMedicineData = "";
  for (int i = 0; i < EEPROM_SIZE; i++) {
    char ch = EEPROM.read(i);
    if (ch == '\0') break;
    storedMedicineData += ch;
  }
  Serial.print("Loaded Stored Medicine Data: ");
  Serial.println(storedMedicineData);
}

void saveStoredMedicineData(String data) {
  Serial.println("Saving Medicine Data to EEPROM...");
  EEPROM.begin(EEPROM_SIZE);
  for (int i = 0; i < data.length(); i++) {
    EEPROM.write(i, data[i]);
  }
  EEPROM.write(data.length(), '\0');
  EEPROM.commit();
  Serial.println("Medicine Data saved successfully.");
}
void setupWebServer() {
  server.begin();
  Serial.println("Web server started");
```

```
    loadStoredMedicineData();
}
void handleCredentials() {
    Serial.println("Received POST request on /credentials");
    String requestBody = server.arg("plain");
    Serial.println("Received Data:");
    Serial.println(requestBody);
    if (requestBody.length() > 0) {
        sortMedicineDataAndStore(requestBody);
    }
    server.send(200, "application/json", "{\"status\": \"success\", \"message\": \"Medicine data
received\"}");
    Serial.println("Response Sent");
}
void sortMedicineDataAndStore(String rawData) {
    DynamicJsonDocument doc(2048);
    DeserializationError error = deserializeJson(doc, rawData);

    if (error) {
        Serial.println("Failed to parse JSON data");
        return;
    }
    JsonArray medicines = doc["medicines"].as<JsonArray>();

    std::sort(medicines.begin(), medicines.end(), [](JsonObject a, JsonObject b) {
        String timeA = a["medicineTimes"][0].as<String>();
        String timeB = b["medicineTimes"][0].as<String>();
        return timeA < timeB;
    });
    String sortedData;
    serializeJson(doc, sortedData);
    saveStoredMedicineData(sortedData);
```

```
    Serial.println("Sorted and Saved Medicine Data");
  }

#endif // WEBSERVER_H
```

7.4.2 MedicineScheduler:

```
#define EEPROM_SIZE 512
String storedMedicineData = "";
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 19800, 60000);
void loadStoredMedicineData();
void syncTime();
String getMedicineToSchedule();
String getCurrentTime();
int convertToMinutes(String timeStr);
String findEarliestUpcomingTime(JsonArray medicineTimes);
void syncTime() {
   timeClient.begin();
   timeClient.update();
}
void loadStoredMedicineData() {
   EEPROM.begin(EEPROM_SIZE);
   storedMedicineData = "";
   for (int i = 0; i < EEPROM_SIZE; i++) {
      char ch = EEPROM.read(i);
      if (ch == '\0') break;
      storedMedicineData += ch;
   }
   Serial.print("Loaded Stored Medicine Data: ");
   Serial.println(storedMedicineData);
}
```

```
String getCurrentTime() {
  timeClient.update();
  unsigned long epochTime = timeClient.getEpochTime();
  int currentHour = (epochTime % 86400L) / 3600;
  int currentMinute = (epochTime % 3600) / 60;
  return String(currentHour) + ":" + (currentMinute < 10 ? "0" : "") + String(currentMinute);
}
int convertToMinutes(String timeStr) {
  int colonIndex = timeStr.indexOf(':');
  int hour = timeStr.substring(0, colonIndex).toInt();
  int minute = timeStr.substring(colonIndex + 1).toInt();
  return (hour * 60) + minute;
}
String findEarliestUpcomingTime(JsonArray medicineTimes) {
  String currentTime = getCurrentTime();
  int currentTimeMinutes = convertToMinutes(currentTime);
  String earliestTime = "";
  int earliestMinutes = 1440;
  for (String medicineTime : medicineTimes) {
    int startIndex = medicineTime.indexOf('T') + 1;
    int endIndex = medicineTime.indexOf('+');
    String extractedTime = medicineTime.substring(startIndex, endIndex);
    int medicineMinutes = convertToMinutes(extractedTime);
    if (medicineMinutes >= currentTimeMinutes && medicineMinutes < earliestMinutes) {
      earliestTime = extractedTime;
      earliestMinutes = medicineMinutes;
    }
  }
  return earliestTime;
}
String getMedicineToSchedule() {
```

```
if (storedMedicineData == "") {
    Serial.println("No medicine data available.");
    return "";
}
DynamicJsonDocument doc(2048);
DeserializationError error = deserializeJson(doc, storedMedicineData);
if (error) {
    Serial.println("Failed to parse stored JSON data.");
    return "";
}


String currentTime = getCurrentTime();
String medicinesToSchedule = "";
const char* daysOfWeek[] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
int currentDayIndex = timeClient.getDay();
String currentDay = daysOfWeek[currentDayIndex];
JsonArray medicines = doc["medicines"].as<JsonArray>();
for (JsonObject medicine : medicines) {
    bool isActive = medicine["isActive"];
    if (!isActive) {
        continue;
    }
   JsonArray selectedDays = medicine["selectedDays"].as<JsonArray>();
    bool isScheduledToday = false;
    for (String day : selectedDays) {
        if (day == currentDay) {
            isScheduledToday = true;
            break;
        }
    }
```

```
      if (!isScheduledToday) {
          continue;
      }
      JsonArray medicineTimes = medicine["medicineTimes"].as<JsonArray>();
      String upcomingTime = findEarliestUpcomingTime(medicineTimes);
      if (upcomingTime != "") {
          JsonArray medicineNames = medicine["medicineNames"].as<JsonArray>();
          for (String medicineName : medicineNames) {
              medicinesToSchedule += medicineName + " at " + upcomingTime + "\n";
          }
      }
  }
  if (medicinesToSchedule == "") {
      return "No medicines scheduled for the current time.";
  }
  return medicinesToSchedule;
}
```

7.4.3 wifi_manager.h:

```
 void WiFiSetup();
void saveWiFiCredentials(int addr, String data);
String readWiFiCredentials(int addr);
void connectToWiFi(const char* ssid, const char* password);
bool isWiFiConfigured();
 void WiFiSetup() {
  Serial.begin(115200);
  EEPROM.begin(EEPROM_SIZE);
  if (isWiFiConfigured()) {
    String savedSSID = readWiFiCredentials(WIFI_SSID_ADDR);
    String savedPassword = readWiFiCredentials(WIFI_PASSWORD_ADDR);
    connectToWiFi(savedSSID.c_str(), savedPassword.c_str());
```

```
  } else {
    WiFi.mode(WIFI_AP_STA);
    WiFi.beginSmartConfig();
    Serial.println("Waiting for SmartConfig.");
    while (!WiFi.smartConfigDone()) {
      delay(500);
      Serial.print(".");
    }

    Serial.println("\nSmartConfig received.");
    Serial.println("Waiting for Wi-Fi.");
    while (WiFi.status() != WL_CONNECTED) {
      delay(500);
      Serial.print(".");
    }
    saveWiFiCredentials(WIFI_SSID_ADDR, WiFi.SSID());
    saveWiFiCredentials(WIFI_PASSWORD_ADDR, WiFi.psk());
    EEPROM.write(WIFI_FLAG_ADDR, 1);
    EEPROM.commit();
    Serial.println("\nWi-Fi Connected.");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
  }
}
void saveWiFiCredentials(int addr, String data) {
  for (int i = 0; i < data.length(); i++) {
    EEPROM.write(addr + i, data[i]);
  }
  EEPROM.write(addr + data.length(), '\0');
  EEPROM.commit();
}
String readWiFiCredentials(int addr) {
  String data = "";
  char character;
  while ((character = EEPROM.read(addr++)) != '\0') {
```

```
    data += character;
  }
  return data;
}
void connectToWiFi(const char* ssid, const char* password) {
  WiFi.begin(ssid, password);
  Serial.println("Connecting to Wi-Fi...");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Connected.");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}
bool isWiFiConfigured() {
  return EEPROM.read(WIFI_FLAG_ADDR) == 1;
}
```

Chapter 8

Conclusion and Result

8.1 Testing:

The Smart Pillbox app is subjected to comprehensive testing across all phases of development to ensure its reliability, accuracy, and security. Various testing strategies are employed to verify that the app meets functional, performance, and user experience standards.

- Unit Testing: Each individual feature such as reminder setting, pill detection, scanning, and report generation is tested in isolation. Functions and modules are validated to ensure they work correctly and consistently according to the specifications.
- Integration Testing: Interconnected features like scanning prescriptions and detecting pills, or configuring settings and receiving reminders, are tested together. This ensures that data flows correctly between modules and that combined functionalities behave as intended.
- System Testing: The complete app is evaluated in a controlled environment to simulate real-world usage. This includes end-to-end testing of medication tracking, caregiver monitoring, pharmacy locator integration, and report generation to verify system-wide behavior.
- User Acceptance Testing (UAT): A pilot group of users, including elderly patients and their

caregivers, tests the app in real-world conditions. Their feedback is used to identify usability improvements, functional issues, and feature enhancements before public release.

- Performance Testing: The app's responsiveness is tested under various conditions, such as high user activity or unstable network environments. Scenarios like multiple concurrent reminders and simultaneous caregiver interactions are analyzed to ensure the app remains efficient and stable.
- Security Testing: Critical security features like encrypted prescription data, secure login credentials, and access controls for caregivers are rigorously tested. The app is checked for vulnerabilities against threats such as unauthorized access, data leakage, and tampering.

8.1.1 Patient User Testcases:

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Type of Testing	Status
TC_P_01	Set a pill reminder	1. Login as patient 2. Navigate to Reminders 3. Set time and medicine	Reminder is saved and user receives notification	Reminder is saved and user receives notification	Functional Testing	Pass
TC_P_02	Scan prescription	1. Login as patient 2. Use Scan Prescription 3. Upload image	System detects medicines and updates info	System detects medicines and updates info	Integration Testing	Pass
TC_P_03	Generate health report	1. Login as patient 2. Go to Reports 3. Tap Generate Report	System generates downloadable report	System generates downloadable report	System Testing	Pass
TC_P_04	Configure dosage settings	1. Login as patient 2. Go to Settings 3. Set dosage configs	Settings saved and reflected in reminders	Settings saved and reflected in reminders	Functional Testing	Pass

TC_P_05	View diagnosis result	1. Login as patient 2. Check Diagnosis tab	Smart diagnosis report is displayed	Smart diagnosis report is displayed	User Acceptance Testing	Pass

Table 8.1.1 Patient User Testcases

8.1.2 Caretaker User Testcases:

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Type of Testing	Status
TC_C_01	Receive medication reminder alerts	1. Login as caregiver 2. Monitor user reminders	Notifications are received as scheduled	Notifications are received as scheduled	Functional Testing	Pass
TC_C_02	View patient health reports	1. Login as caregiver 2. Go to Reports 3. Select patient	Patient's report is displayed	Patient's report is displayed	Integration Testing	Pass
TC_C_03	Coordinate with pharmacy	1. Login as caregiver 2. Locate nearby pharmacy 3. Send request	Pharmacy receives and responds to request	Pharmacy receives and responds to request	System Testing	Pass
TC_C_04	Assist in smart diagnosis	1. Login as caregiver 2. Go to Diagnosis 3. Review diagnosis results	Can view and annotate smart diagnosis data	Can view and annotate smart diagnosis data	User Acceptance Testing	Pass

Table 1 Caretaker User Testcases

8.2 Future Scope:

1. AI-Powered Health Insights: Incorporate machine learning algorithms to provide personalized medicine suggestions and detect irregular patterns in medicine intake or health conditions.
2. Integration with Wearables: Support synchronization with wearable devices like

smartwatches to enhance health tracking and real-time notifications.

3. Voice Assistant Support: Add voice-controlled interaction for elderly users to make the system more accessible and user-friendly.
4. Global Pharmacy Integration: Enable access to a global network of pharmacies for medication availability, price comparison, and automated ordering.
5. Telemedicine Features: Include a module for virtual consultations with doctors based on the reports and diagnosis generated by the app.

8.3 Limitations:

1. Hardware Dependency: The effectiveness of pill detection and reminders relies heavily on compatible hardware and sensors.
2. Internet Connectivity: Some features like cloud sync, pharmacy requests, and report sharing require constant internet access.
3. Prescription Scanning Accuracy: OCR-based prescription scanning might misinterpret handwritten notes or poorly scanned documents.
4. User Privacy Risks: Handling sensitive health data may expose the system to privacy and security concerns if not managed properly.
5. Limited Language Support: The app might not support all regional languages initially, which can affect accessibility for a diverse user base.

8.4 Conclusion:

The Smart Pillbox System significantly enhances medication management through timely reminders, accurate pill detection, smart diagnosis, and effective caregiver coordination. It simplifies the process for both patients and caregivers while maintaining detailed health records and improving medication adherence. Although it has some limitations, the system lays a strong foundation for future development in digital health management, aiming for increased automation, intelligence, and accessibility.

References

https://pmc.ncbi.nlm.nih.gov/articles/PMC6045499/

https://en.wikipedia.org/wiki/C%2B%2B

https://reviewsreporter.com/scope-of-c-plus-plus-course/

https://beecrowd.com/blog-posts/best-python-courses/

https://solguruz.com/blog/flutter-for-hybrid-apps/

https://www.twitchtime.com/blogs/kotlin-android-app-development/

https://pmc.ncbi.nlm.nih.gov/articles/PMC9618185/

https://www.researchgate.net/publication/304630433_A_comparison_of_automated_dispensing_cabinet_optimization_methods

https://www.researchgate.net/publication/388291692_Smart_Pill_Box_An_IoT-Integrated_Application_for_Monitoring_Patient_Medication_Usage_at_Home

https://pmc.ncbi.nlm.nih.gov/articles/PMC8521858/

https://www.youtube.com/watch?v=avrdDZD7qEQ

https://medium.com/@nitishk72/flutter-local-notification-1e43a353877b

https://medium.com/calmexperts/implementing-kotlin-native-code-on-your-flutter-application-b576ab524b41

Acknowledgement

The success of this project would not have been possible without constant advice, encouragement and support from a vast number of people.

We, the members of the team who developed “Smart Pillbox” take immense pleasure in thanking Professor Dr. Mohd. Zafar Shaikh, Principal of Shri Bhagubhai Mafatlal Polytechnic and College of Engineering for having permitted us to carry out this project work. We also thank our H.O.D Mr. Janardan Kulkarni for leading us the advice and helping us in our needs for making our project more successful.

We wish to express our deep sense of gratitude to our Internal Guide, Mrs. Sharyu Kadam for her constant guidance and useful suggestions, which helped us in completing the project work in time. We would like to thank her for her valuable assistance in the project work. A Big thanks to Mr. Sachin Talekar for looking after our needs in the lab, being patient with our continual equipment demands, and his encouraging support. Words are inadequate in offering our thanks to the entire staff of C.S.E for providing us with all amenities and facilities.

Finally, yet importantly, we would like to express our heartfelt thanks to our beloved parents for their blessings, our friends for their help and wishes for the successful completion of this project.